

EXPCONCAD: Experience-Guided Text-to-CAD Generation from Shape Descriptions with Implicit Spatial Constraints

Jingyao Liu^{1,2} Jinkang Tang^{1,2} Chen Huang^{1,2,3} Wenqiang Lei^{1,2*} See-Kiong Ng³

¹College of Computer Science, Sichuan University

²Engineering Research Center of Machine Learning and Industry Intelligence,
Ministry of Education, China

³Institute of Data Science, National University of Singapore
liujingyao1@stu.scu.edu.cn

Abstract

Text-to-CAD aims to generate executable CAD programs from natural-language descriptions. However, real-world descriptions are often underspecified and omit critical spatial constraints required for valid CAD construction, a challenge that has been largely overlooked by existing methods. In this paper, we argue that missing spatial constraints should be inferred with respect to the underlying construction structure and informed by reusable design experience. Based on this insight, we propose EXPCONCAD, an experience-enhanced framework for implicit spatial constraint completion. EXPCONCAD first recovers the intended construction structure and constraint scopes, then retrieves relevant constraint-completion experience for similar scopes to complete the missing spatial constraints, and finally generates executable CadQuery programs. Extensive experiments demonstrate the effectiveness of EXPCONCAD and provide insights into the role of construction structure understanding and experience memory in spatial constraint completion. Our code is available at: <https://github.com/Hotjiashell/ExpConCAD>.

1 Introduction

Automatically translating natural-language design descriptions into executable CAD construction programs, such as CadQuery scripts (Contributors, 2026), allows users to create 3D models directly from textual instructions without manually operating professional CAD software (Khan et al., 2024; Li et al., 2024). Recent advances in large language models (LLMs) (OpenAI, 2025; Yang et al., 2025; Qwen Team, 2026) have substantially improved this Text-to-CAD paradigm (Badagabettu et al., 2024; Wang et al., 2026). As shown in Figure 1(a), given a detailed specification of a target object, the LLM generates the Python code, which is then executed to construct the corresponding 3D geometry.

However, existing Text-to-CAD methods typically require users to explicitly specify detailed geometric parameters, modeling operations, and spatial relations of the target object (Khan et al., 2024; Li et al., 2024; Liao et al., 2025; Kolodiazhnyi et al., 2025; Guan et al., 2026; Govindarajan et al., 2025). While such fine-grained inputs help define precise spatial constraints among object components, specify such over-detailed instructions impose substantial burden on users. In realistic scenarios, users usually describe the desired shape at a much higher semantic level. For example, as shown in Figure 1(b), a user may request “a flat plate with two circular through holes”. Such descriptions convey the intended appearance of the object, while leaving much of the underlying spatial constraints implicit. As a result, existing methods often generate structurally incorrect CAD models, such as overlapping holes. Recent study attempts to address this under-specification through end-to-end optimization by directly aligning under-specified inputs with target CAD shapes (Wang et al., 2025a). However, it only learns input-output correlations while overlooking the underlying implicit spatial constraints. Consequently, when encountering unseen designs with different spatial arrangements or component interactions, it still struggles to generalize and fails to infer the missing constraints for accurate CAD generation. Therefore, **effectively reasoning about implicit spatial constraints in under-specified instructions remains a critical challenge for generating accurate 3D CAD structures.**

To address this challenge, we start from a basic property of CAD construction: building a valid CAD model first requires identifying geometric elements and modeling operations, and then specifying the spatial relations among them (Kyratzi and Azariadis, 2021; Camba et al., 2016). Thus, for underspecified shape descriptions with implicit constraints, the missing spatial relations should be com-

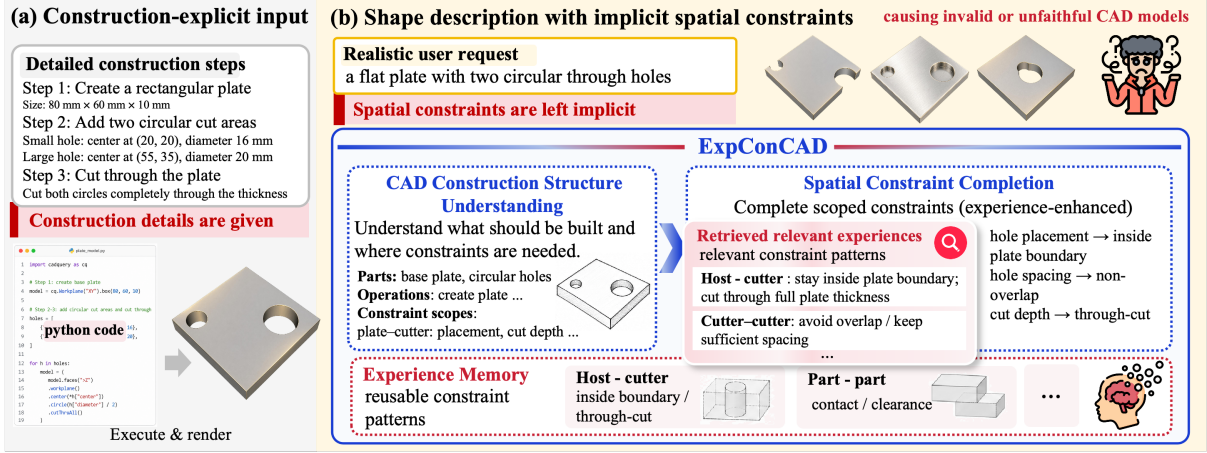


Figure 1: Overview of EXPCONCAD for resolving implicit spatial constraints (Fig.a and Fig.b). EXPCONCAD first recovers the construction structure to identify constraint scopes, and then completes missing spatial constraints with guidance from transferable experience.

pleted with respect to the underlying construction structure and its constraint scopes, rather than inferred from surface descriptions alone. We further observe that similar constraint-completion problems repeatedly appear across historical CAD construction processes (Kyratzi and Azariadis, 2021; Camba et al., 2016). For example, a cylindrical part with side holes and a plate with through holes have different target geometries, but both involve comparable constraint scopes, such as cutter-base placement and cutter-cutter separation. This suggests that prior CAD cases can provide transferable experience, such as cutter containment, cutter separation, and through-cut depth, for completing missing constraints over similar scopes. Inspired by these observations, we propose **EXPCONCAD** (**Ex**perience-enhanced **Con**straint completion for **CAD**), a framework for generating faithful CAD programs from underspecified descriptions by explicitly completing implicit spatial constraints. EXPCONCAD couples **construction-aware spatial constraint reasoning** with an **experience memory**. The reasoning pipeline first performs *Construction Structure Understanding* to recover the intended construction process and identify constraint scopes, then performs *Spatial Constraint Completion* before generating executable CadQuery code. The experience memory distills reusable constraint-completion patterns from prior CAD cases and retrieves relevant experience to guide completion over similar scopes.

We evaluate EXPCONCAD on CADFUSION-HARD, a challenging benchmark for Text-to-CAD with underspecified spatial constraints. EXPCONCAD improves VLM-Score over the strongest

baseline by 22.5% on Qwen3.5-27B and 27.9% on GPT-5. Further analyses show that Construction Structure Understanding provides reliable scopes for Spatial Constraint Completion, while experience memory is most effective when it stores reusable scoped constraint-completion patterns. Our contributions are three-fold:

- We highlight the *implicit spatial constraint* as a critical challenge in Text-to-CAD, where natural descriptions often omit spatial relations required for valid CAD construction.
- We propose EXPCONCAD, which combines construction-aware spatial constraint reasoning and reusable experience memory from prior CAD cases for improved CadQuery generation.
- We conduct extensive experiments to validate our effectiveness and characteristics.

2 Related Work

CAD modeling is essential for mechanical design and digital manufacturing, but creating CAD models typically requires domain expertise and familiarity with professional modeling software (Cherng et al., 1998; Shah and Mäntylä, 1995; Bhavnani et al., 2001; Chester, 2007). Text-to-CAD aims to lower this barrier by converting natural-language design descriptions into structured CAD representations. Many existing methods represent CAD models as parametric command sequences (Wu et al., 2021; Khan et al., 2024; Li et al., 2024; Liao et al., 2025; Wang et al., 2025b), which often require dedicated interpreters or reconstruction procedures to recover CAD models. Recent work has therefore shown growing interest in generating executable

Method	ISC	Input	Output	Core Idea
Text2CAD (Khan et al., 2024)	✗	CED	Seq.	Construction-explicit sequence generation.
CADRille (Kolodiazhnyi et al., 2025)	✗	CED	CQ	Construction-explicit CadQuery generation.
CAD-Coder (Guan et al., 2026)	✗	CED	CQ	CoT-based operation planning for CadQuery generation.
CADFusion (Wang et al., 2025a)	✗	USD	Seq.	Visual supervision for shape alignment.
EXPCONCAD	✓	USD	CQ	Experience-guided spatial constraint completion.

Table 1: **Comparison of representative Text-to-CAD methods.** ISC indicates whether the method explicitly addresses implicit spatial constraint completion. CED: construction-explicit descriptions; USD: underspecified shape descriptions with implicit spatial constraints; Seq.: CAD command sequence; CQ: CadQuery code.

CAD programs, such as CadQuery scripts (contributors, 2026; Xie and Ju, 2025; Kolodiazhnyi et al., 2025; Guan et al., 2026), which is executed to create CAD models. Following this direction, we focus on generating executable CadQuery programs from natural-language descriptions.

However, most Text-to-CAD methods assume that the input description explicitly specifies the information required for CAD construction, including modeling operations, geometric parameters, and spatial relations. Under this assumption, both task-specific models (Khan et al., 2024; Li et al., 2024; Liao et al., 2025; Xie and Ju, 2025; Kolodiazhnyi et al., 2025; Guan et al., 2026) and recent LLM-based approaches (Alrashedy et al., 2025; Yuan et al., 2026; Schüpbach et al., 2025; Li et al., 2025) primarily focus on translating detailed user instructions into target CAD representations. However, real-world descriptions are often underspecified and omit critical spatial constraints required for CAD construction. As a result, generating executable CAD programs from such descriptions requires not only translation, but also inferring the missing constraints. Only limited work has explored this problem. CADFusion (Wang et al., 2025a) leverages visual supervision to improve shape consistency, but it mainly aligns generated shapes with textual descriptions rather than explicitly reasoning about the missing spatial constraints. Consequently, it struggles to infer implicit constraints and generalize to underspecified inputs. Therefore, completing missing spatial constraints from incomplete natural-language descriptions remains a largely unexplored challenge.

3 EXPCONCAD

Problem Formulation and Overview. Given an underspecified inputs x , our goal is to generate an executable CadQuery program y . The central challenge is to complete the implicit spatial constraints

that are omitted from x but are required for valid CAD construction. As shown in Figure 2, EXPCONCAD addresses this challenge through two key designs: (a) a construction-aware spatial constraint reasoning pipeline, and (b) an experience memory $\mathcal{B}$ that contains reusable guidance for constraint completion. The reasoning pipeline first recovers the intended construction process and identifies the scopes where spatial constraints should be completed. It then performs spatial constraint completion with the support of retrieved experience $\mathcal{R}$ from $\mathcal{B}$. Finally, the completed constraints are used to generate the executable CadQuery program.

3.1 Construction-Aware Spatial Constraint Reasoning

To turn the under-specified descriptions into valid CadQuery programs, as shown in Figure 2(a), EXPCONCAD involves the following three steps:

Construction Structure Understanding (CSU).

This step recovers the construction process implied by the input and identifies the scopes for subsequent spatial constraint completion:

$$s = L_{\text{CSU}}(x). \quad (1)$$

The recovered structure s includes the CAD elements, modeling operations, and constraint scopes where missing spatial relations should be completed. A constraint scope specifies where a missing constraint should hold, such as between an operation and its target object or between two geometric elements. For example, for “*a plate with two circular through holes*”, CSU identifies the base plate, two cylindrical cutters, and two Boolean-cut operations, and derives scopes such as cutter–plate placement and cutter–cutter separation from these elements, operations, and the design intent.

Spatial Constraint Completion (SCC). This step infers the missing spatial relations within the

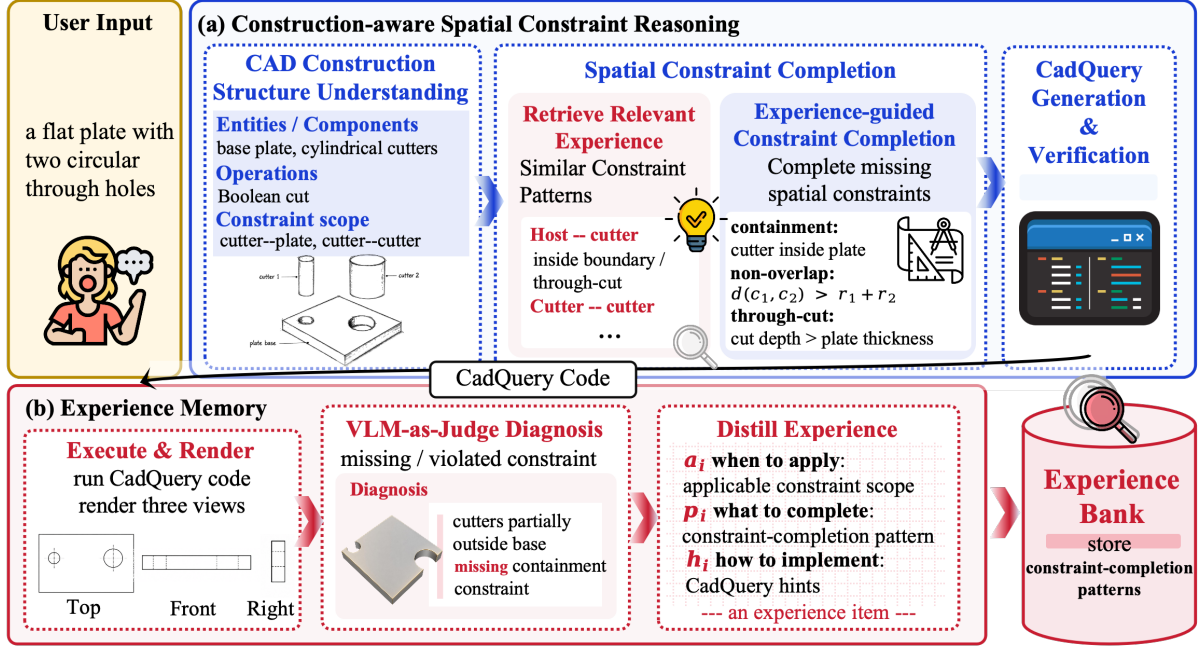


Figure 2: **Overview of EXPCONCAD.** (a) The framework identifies constraint scopes through construction-structure recovery and completes implicit spatial constraints before CadQuery generation. (b) Experience memory stores reusable constraint-completion patterns from prior CAD cases for future retrieval.

identified scopes to support valid CAD generation:

$$c = L_{SCC}(x, s). \quad (2)$$

The completed constraint set c specifies how the identified elements and operations should be spatially configured, including coordinate conditions, distance bounds, size relations, and operation parameters. For example, for “a plate with two circular through holes”, SCC instantiates the missing constraints as parameter-level conditions: each cutter center should lie within the plate boundary, the distance between cutter centers should exceed the sum of their radii, and the cut depth should be larger than the plate thickness. Section 3.2 describes how retrieved experience facilitates this process.

CadQuery Generation. After completing the spatial constraints, EXPCONCAD generates the final executable program:

$$y = L_{Code}(x, s, c). \quad (3)$$

The code generator conditions on the original description x , the recovered construction structure s , and the completed constraint set c to produce executable CadQuery code. To ensure that the program can produce a CAD model, we execute it for compilation and rendering checks. If execution fails, only the code is revised according to the error message, while s and c remain fixed to preserve the completed spatial reasoning.

3.2 Experience Memory for Spatial Constraint Completion

We construct a constraint-completion experience memory from prior CAD cases to enhance SCC. For instance, repeated side-hole constructions reveal reusable constraint patterns, including cutter containment, hole non-overlap, and sufficient cutting depth. EXPCONCAD distills these patterns into reusable experience and retrieves relevant memories to guide constraint completion in analogous construction scopes.

Experience Format. To guide constraint completion over similar constraint scopes, each experience item should be both retrievable and actionable. Therefore, we design each item to answer three questions: when it should be applied, what constraint-completion pattern it provides, and how the pattern can be implemented in CadQuery. Formally, each item is represented as

$$e_i = (a_i, p_i, h_i), \quad (4)$$

where a_i specifies the applicable constraint scope used for experience retrieval, including the involved CAD elements and operations; p_i describes the reusable constraint-completion pattern, such as containment, non-overlap, symmetry, or through-cut; and h_i records how to implement these constraints in CadQuery, such as parameter bounds,

placement rules, coordinate conditions, or Boolean-operation settings. For example, a cutter–plate experience item may use a_i to specify a hole-cutting scope between a cutter and a base solid, p_i to describe boundary-containment and through-cut constraints, and h_i to record CadQuery implementation hints such as valid coordinate bounds and cut-depth settings.

Experience Construction. We construct experience items from failed prior generations, which reveal missing or violated spatial constraints that the model has not yet handled. For each generated CadQuery program y_i , we execute the code and render the resulting CAD model into views v_i . Given the description x_i and rendered views v_i , a VLM-as-judge diagnoses constraint-related errors:

$$d_i = L_{\text{Judge}}(x_i, v_i). \quad (5)$$

The diagnosis focuses on missing or violated spatial constraints, such as overlapping holes, cutters outside the base, or cuts that do not pass through the object. We distill experience only from diagnosed failure cases, using the diagnosis and the corresponding implementation context:

$$e_i = L_{\text{Distill}}(d_i, y_i) = (a_i, p_i, h_i). \quad (6)$$

For example, if a plate with two through holes contains overlapping holes, the failure can be distilled into a cutter–cutter experience item for non-overlap, with center-distance bounds as implementation hints. If a cutter is placed outside the plate or fails to cut through it, the failure can be distilled into a cutter–plate item for boundary containment or through-cut depth. This turns instance-level failures into reusable experience to guide constraint completion in analogous construction scopes.

Experience Use. Experience memory enhances the basic SCC process in Eq. 2 with relevant experience from similar constraint scopes. During inference, given x and the recovered construction structure s , we construct fine-grained queries over the identified constraint scopes:

$$Q = \{q_j\}_{j=1}^m = L_{\text{Query}}(x, s). \quad (7)$$

Each query corresponds to a scope where constraints may be missing, such as cutter–plate placement or cutter–cutter separation in the plate example. We retrieve relevant experience by matching

each query with the applicable construction situation a_i of each memory item:

$$\mathcal{R} = \bigcup_{j=1}^m \text{TopK}(q_j, \mathcal{B}). \quad (8)$$

This retrieves experience from similar constraint scopes rather than whole CAD cases. Spatial Constraint Completion is then enhanced by conditioning on both the current construction structure and retrieved experience:

$$c = L_{\text{SCC}}(x, s, \mathcal{R}). \quad (9)$$

Thus, the memory provides targeted guidance for completing constraints within the current scopes.

4 Experiments

4.1 Experimental Setup

Benchmarks and Experience Construction. We report main results on CADFUSION-HARD, the more challenging subset of CADFusion (Wang et al., 2025a) with underspecified shape descriptions. Full CADFusion results and CADFUSION-HARD construction details are provided in Appendices D.2 and C.1. Following prior experience-based settings (Cai et al., 2025; Ouyang et al., 2026), we construct the experience bank from randomly sampled training examples, keeping all evaluation samples disjoint. We report the strongest experience-enhanced result for each backbone and provide a detailed analysis of how experience-bank size affects performance in Section 4.4. Details are in Appendix C.2.

Baselines and Backbones. We compare with baselines covering both input-description settings discussed in Section 2. The baselines include trained Text-to-CAD models, i.e., Text2CAD (Khan et al., 2024), CADrille (Kolodiazhnyi et al., 2025), CAD-Coder (Guan et al., 2026), and CADFusion (Wang et al., 2025a), as well as LLM-based methods, including Vanilla, CoT, and the agentic baseline CadCodeVerify (Alrashedy et al., 2025). We also report Ours w/o Exp. to isolate the effect of experience. For LLM-based methods, since some baselines require both textual and rendered-view inputs, we use two multimodal LLMs from different model families, Qwen3.5-72B and GPT-5, as backbones. Implementation details are provided in Appendix C.4.

Metrics. Following prior Text-to-CAD evaluations (Khan et al., 2024; Wang et al., 2025a),

Method	CadFusion-Hard							
	VLM $\uparrow$	m. r-IoU $\uparrow$	md. r-IoU $\uparrow$	m. CD $\downarrow$	md. CD $\downarrow$	m. rCD $\downarrow$	md. rCD $\downarrow$	Inv. $\downarrow$
<i>Trained Text-to-CAD Models</i>								
Text2CAD	1.88	0.182	0.140	104.983	90.268	70.634	51.074	0.00
Cadrille	1.17	0.042	0.021	156.687	142.039	83.456	74.495	5.00
CAD-Coder	1.54	0.283	0.248	67.414	51.511	39.109	27.750	3.00
CadFusion	4.22	0.287	0.230	65.582	67.797	31.310	26.268	23.50
<i>Qwen3.5-27B</i>								
Vanilla	3.33	0.233	0.112	88.289	123.459	39.586	54.98	50.50
CoT	3.05	0.206	0.133	102.622	127.411	42.674	52.875	67.00
CADCodeVerify	5.11	0.324	0.300	59.125	42.948	26.607	18.654	13.50
Ours w/o Exp.	<u>5.63</u>	<u>0.342</u>	0.333	50.264	32.729	<u>22.484</u>	14.435	<u>1.00</u>
Ours	6.26	0.343	<u>0.313</u>	<u>51.952</u>	<u>34.638</u>	21.925	<u>14.525</u>	2.00
<i>GPT-5</i>								
Vanilla	4.09	0.276	0.195	84.995	103.535	34.609	33.499	38.50
CoT	4.05	0.284	0.229	81.875	69.961	28.787	25.695	32.00
CADCodeVerify	5.16	0.326	0.281	62.673	47.859	26.449	20.651	21.00
Ours w/o Exp.	<u>6.14</u>	<u>0.361</u>	<u>0.348</u>	<u>50.178</u>	<u>32.568</u>	<u>20.514</u>	<u>13.348</u>	<u>0.50</u>
Ours	6.60	0.371	0.351	47.496	29.487	19.431	11.122	<u>0.50</u>

Table 2: Main results on CADFUSION-HARD. Best and second-best results are shown in **bold** and underlined. Non-renderable outputs are included with worst-decile scores estimated from valid outputs.

we report VLM-Score (VLM), rotation-invariant Intersection-over-Union (r-IoU), Chamfer Distance (CD), rotation-invariant Chamfer Distance (rCD), and Invalid Ratio (Inv.). VLM measures text-shape consistency; r-IoU, CD, and rCD measure geometric quality via volumetric overlap and point-cloud distance; Inv. measures generation validity. For r-IoU, CD, and rCD, we report both mean (m.) and median (md.) values. Since these metrics are typically computed only on successfully rendered outputs, large Inv. differences in the main comparison may unfairly favor methods with many invalid CAD models. We therefore include non-renderable outputs in the main aggregation using a worst-decile value estimated from valid outputs, motivated by validity-aware evaluation (Mallis et al., 2026). For analysis experiments, where Inv. differences are less pronounced, we use standard rendered-only computation. (Appendices D.1, C.5, and C.3)

Implementation Details. All L . modules are implemented with LLM prompting. Detailed prompts are provided in Appendix A.

4.2 Main Results

As shown in Table 2, EXPCONCAD achieves the highest VLM scores on both Qwen3.5-27B and

GPT-5, indicating superior alignment with user descriptions. Notably, even without experience memory, EXPCONCAD substantially outperforms existing methods, demonstrating that construction-aware spatial constraint reasoning can effectively recover construction structures and infer implicit spatial constraints from underspecified descriptions. Incorporating experience memory further improves VLM by 11.2% on Qwen3.5-27B and 7.5% on GPT-5, suggesting that reusable experience provides additional guidance for resolving missing constraints and better capturing user intent. Consistent qualitative evidence is shown in Figure 3, where EXPCONCAD generates CAD models that more closely resemble the ground truth. For geometry-based metrics, EXPCONCAD variants also achieve the strongest overall performance, although the gains from experience memory are less consistent. A possible explanation is that underspecified descriptions may correspond to multiple valid geometries. Consequently, overlap- and distance-based metrics can be sensitive to acceptable geometric variations even when the generated CAD model satisfies the intended design constraints. Overall, the results show that *our spatial constraint reasoning is the primary driver of performance, while experience memory further improves*

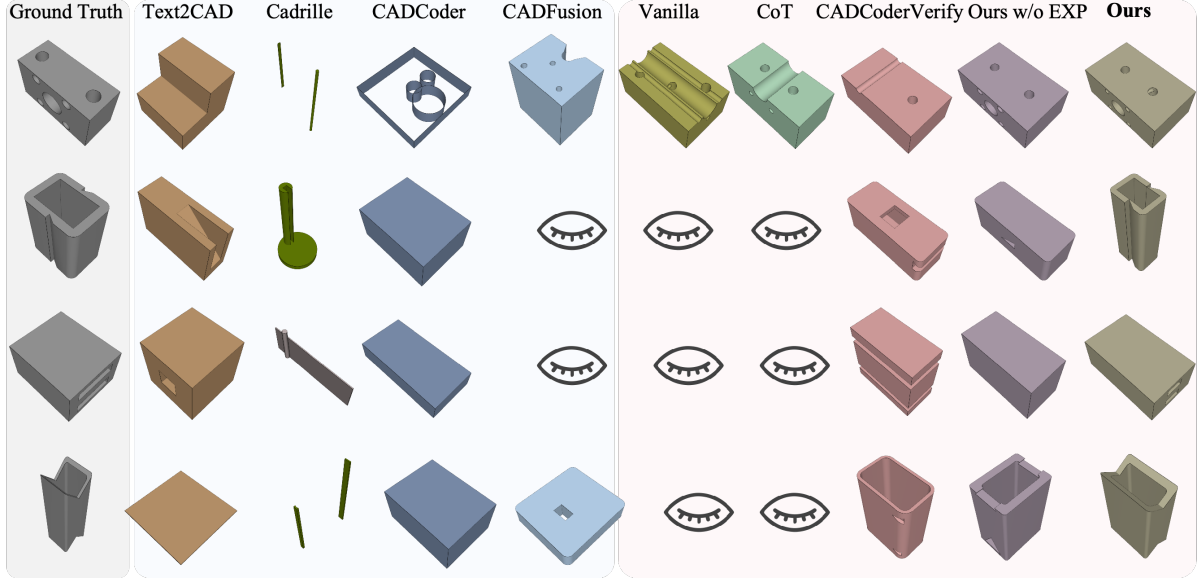


Figure 3: **Qualitative comparison on representative hard cases.** Each row compares the ground-truth CAD model with outputs from different methods. All LLM-based methods use Qwen3.5-27B as the backbone. The closed-eye symbol denotes a rendering failure. See Appendix D.5 for input descriptions and generation traces.

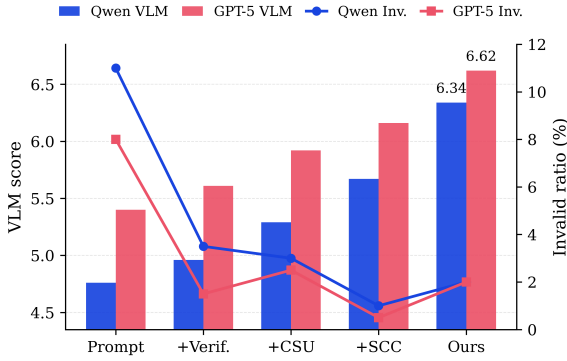


Figure 4: Ablation study.

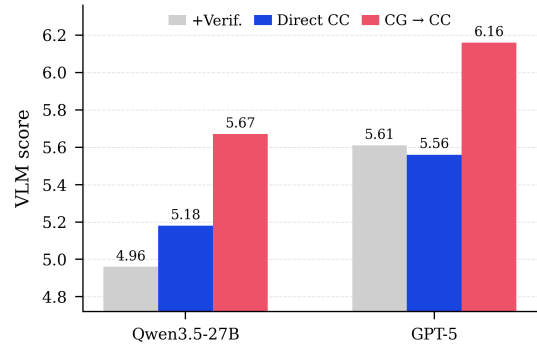


Figure 5: CSU as a prerequisite for SCC.

constraint completion and description fidelity.

4.3 Ablation Study

We conduct an ablation study to isolate the contribution of each major component in EXPCONCAD. As shown in Figure 4, *Prompt* denotes a strong task-specific prompt, *+Verif.* adds execution-based verification and revision, *+CSU* introduces Construction Structure Understanding, *+SCC* adds Spatial Constraint Completion, and *Ours* further uses experience-enhanced SCC. Adding verification substantially reduces Inv.. Construction Structure Understanding and Spatial Constraint Completion then progressively improve VLM by identifying constraint scopes and completing missing spatial constraints. The full framework achieves the best VLM score, showing that experience

memory further improves constraint completion. These results confirm that EXPCONCAD’s gains come from combining executable-code verification, construction-aware spatial constraint reasoning, and experience-enhanced SCC. Full metric results are provided in Appendix D.4, Table 7.

4.4 Analysis of Key Design Choices

This section analyzes the effectiveness of EXPCONCAD by addressing two questions: (1) Why is the CSU required before the SCC? and (2) How does experience memory facilitate the SCC? Detailed experimental settings are provided in Appendix C.5.

Does CSU provide a better basis for SCC? To understand this, we perform analysis under three settings: a verification-only baseline, direct SCC from underspecified descriptions, and CSU followed by

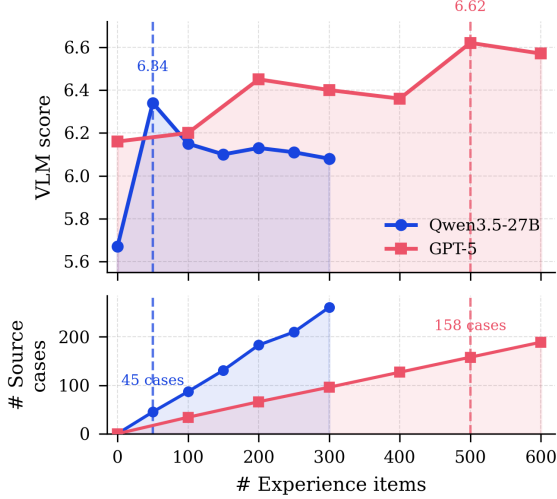


Figure 6: Effect of experience memory on SCC.

SCC. As shown in Figure 5¹, direct SCC yields limited and unstable gains, whereas CSU+SCC consistently improves performance across both backbones. This suggests that underspecified descriptions alone can not specify the constraint scopes to be completed. By first recovering construction elements, operations, and constraint scopes, CSU provides SCC with explicit completion targets. Therefore, missing spatial constraints are more effectively inferred from the recovered construction structure than from the original description alone.

What makes experience useful for SCC? Figure 6 shows that experience improves SCC, but simply adding more experience does not always help. For Qwen3.5-27B, performance peaks with 50 experience items and then saturates, suggesting that the added items are not always reusable for the current constraint-completion need. A likely reason is that its experience often remains case-level: one item may mix several constraints from the original CAD case, making it hard to match a specific scope in a new case. In contrast, GPT-5 benefits more from a larger memory. Its experience is more often distilled into scope-specific patterns, such as cutter-base placement, cutter-cutter separation, or through-cut depth. These patterns are easier to retrieve and reuse when a new case encounters a similar constraint scope. Cases in Appendix D.5 provide examples. Overall, the key is not to store more cases, but to store reusable constraint-completion patterns at the right scope.

¹Full results are reported in Appendix D.4, Table 7.

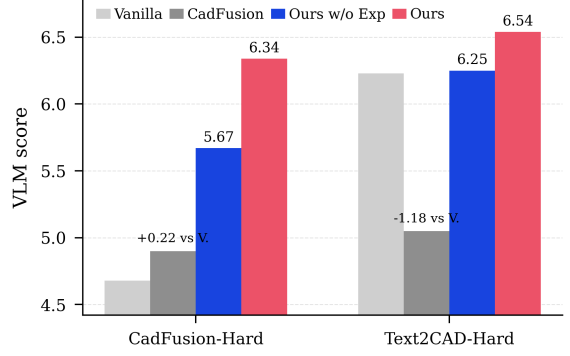


Figure 7: Robustness and experience transfer on unseen cases.

4.5 Robustness and Experience Transfer

We further evaluate the robustness of EXPCONCAD and the transferability of its experience on unseen cases. Using Qwen3.5-27B Vanilla as the reference baseline, we compare EXPCONCAD with CadFusion on CadFusion-Hard and Text2CAD-Hard, where the latter follows a different shape distribution. (details in Appendix C.5). As shown in Figure 7², EXPCONCAD improves over Vanilla on both datasets, with further gains from experience even on unseen Text2CAD-Hard cases. By contrast, CadFusion improves on CadFusion-Hard but drops below Vanilla on Text2CAD-Hard. These results suggest that construction-structure recovery and spatial constraint completion form a robust pipeline, while experience memory transfers through recurring constraint-completion patterns rather than source-dataset shape memorization.

5 Conclusion

In this work, we studied Text-to-CAD generation under underspecified descriptions, where missing spatial constraints hinder accurate CAD construction. To address this challenge, we proposed EXPCONCAD, which combines construction-aware spatial constraint reasoning with an experience memory. More broadly, our work highlights a fundamental shift for CAD generation: from reconstructing geometry explicitly described by users to inferring the design knowledge that users leave unstated. We believe this ability to reason about missing design intent will be essential for building more human-centric CAD agents or copilots that collaborate naturally with human designers.

²Full results are provided in Appendix D.3, Table 6.

Limitations

A key limitation of this work concerns experience management. Although the experience memory is constructed from prior CAD construction cases and used to guide spatial constraint completion, we do not study how it should be updated, deleted, merged, or validated over time. This is important because CAD constraint-completion patterns are often recurring and finite, so continuously expanding the memory may increase storage and retrieval costs without adding genuinely new patterns. In addition, multiple experiences may instantiate the same pattern but differ in quality or executability. Future work should develop memory management strategies to retain high-quality constraint-completion patterns, merge redundant experiences, and keep the memory compact and reliable.

Ethical Considerations

This work aims to make CAD generation more accessible by enabling users to create CAD models from natural-language descriptions, supporting rapid prototyping, education, and design assistance. The study does not involve human subjects, sensitive personal data, or privacy-sensitive information, and all source data come from open-source communities. Human annotation and verification, where used, were conducted by internal CS Master’s and PhD students with CAD-related coursework or experience; participation was voluntary, and written instructions were provided. Details are given in Appendix C.3. We follow the ACL Code of Ethics, and to the best of our knowledge, this study does not raise major concerns related to privacy, bias, or safety.

Reproducibility To facilitate reproducibility, we will publicly release the source code on GitHub (<https://anonymous.4open.science/r/ExpConCAD-B118>). Detailed setup and usage instructions are provided in Appendix A and in the repository README.

Artifact License and Usage Terms All code, data, and artifacts used or created in this work comply with their respective licenses and usage terms. External code and datasets are open-source or publicly available, and we respect their licenses, including proper attribution to the original creators. The artifacts created by EXPCONCAD are released under CC-BY 4.0 and are intended for research purposes.

LLM Usage LLMs are used in this work as the backbone models of EXPCONCAD, as described in Appendix C. In addition, LLMs were used only for language polishing during paper writing. All method design, experiments, analysis, and core ideas were developed solely by the authors.

References

- Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Haider Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. 2025. Generating cad code with vision-language models for 3d designs. In *International Conference on Learning Representations*, volume 2025, pages 52236–52262.
- Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. 2024. Query2cad: Generating cad models using natural language queries. *arXiv preprint arXiv:2406.00144*.
- Suresh K Bhavnani, Frederick Reif, and Bonnie E John. 2001. Beyond command knowledge: Identifying and teaching strategic knowledge for using complex computer applications. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 229–236.
- Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, and 1 others. 2025. Training-free group relative policy optimization.
- Jorge D Camba, Manuel Contero, and Pedro Company. 2016. Parametric cad modeling: An analysis of strategies for design reusability. *Computer-aided design*, 74:18–31.
- John G Cherng, Xin-Yu Shao, Yubao Chen, and Peter R Sferro. 1998. Feature-based part modeling and process planning for rapid response manufacturing. *Computers & industrial engineering*, 34(2):515–530.
- Ivan Chester. 2007. Teaching for cad expertise. *International journal of technology and design education*, 17(1):23–35.
- CadQuery contributors. 2026. *Cadquery*.
- John H Conway, Heidi Burgiel, and Chaim Goodman-Strauss. 2016. *The symmetries of things*. CRC Press.
- Prashant Govindarajan, Davide Baldelli, Jay Pathak, Quentin Fournier, and Sarath Chandar. 2025. Cadmium: Fine-tuning code language models for text-driven sequential cad design. *arXiv preprint arXiv:2507.09792*.
- Yandong Guan, Xilin Wang, Ximing Xing, Jing Zhang, Dong Xu, and Qian Yu. 2026. Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward. *Advances in Neural Information Processing Systems*, 38:59765–59789.

- Mohammad S Khan, Sankalp Sinha, Talha U Sheikh, Didier Stricker, Sk A Ali, and Muhammad Z Afzal. 2024. Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts. *Advances in Neural Information Processing Systems*, 37:7552–7579.
- Maksim Kolodiaznyi, Denis Tarasov, Dmitrii Zhemchuzhnikov, Alexander Nikulin, Ilya Zisman, Anna Vorontsova, Anton Konushin, Vladislav Kurenkov, and Danila Rukhovich. 2025. cadrille: Multi-modal cad reconstruction with online reinforcement learning. *arXiv preprint arXiv:2505.22914*.
- Sofia Kyratzi and Philip Azariadis. 2021. A constraint-based framework to recognize design intent during sketching in parametric environments. *Computer-Aided Design & Applications*, 18(3).
- Xingang Li, Yuewan Sun, and Zhenghui Sha. 2025. Llm4cad: Multimodal large language models for three-dimensional computer-aided design generation. *Journal of Computing and Information Science in Engineering*, 25(2):021005.
- Xueyang Li, Yu Song, Yunzhong Lou, and Xiangdong Zhou. 2024. Cad translator: An effective drive for text to 3d parametric computer-aided design generative modeling. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 8461–8470.
- Jianxing Liao, Junyan Xu, Yatao Sun, Maowen Tang, Sicheng He, Jingxian Liao, Shui Yu, Yun Li, and Xiaohong Guan. 2025. Automated cad modeling sequence generation from text descriptions via transformer-based large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21720–21748.
- Dimitrios Mallis, Marco Wang, Ahmet Serdar Karadeniz, Elisa Ricci, Anis Kacem, and Djamila Aouada. 2026. Text-to-cad evaluation with cadtests. *arXiv preprint arXiv:2605.07807*.
- OpenAI. 2025. Introducing gpt-5. <https://openai.com/index/introducing-gpt-5/>. Accessed: 2026-05-19.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. Reasoning-bank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*.
- Qwen Team. 2026. Qwen3.5: Towards native multi-modal agents.
- Aurel Schüpbach, Raul San Miguel, Julian Ferchow, and Mirko Meboldt. 2025. From text to design: a framework to leverage llm agents for automated cad generation. *Proceedings of the Design Society*, 5:1893–1902.
- Jami J Shah and Martti Mäntylä. 1995. *Parametric and feature-based CAD/CAM: concepts, techniques, and applications*. John Wiley & Sons.
- Pere-Pau Vázquez, Miquel Feixas, Mateu Sbert, and Wolfgang Heidrich. 2001. Viewpoint selection using viewpoint entropy. In *VMV*, volume 1, pages 273–280.
- Liang Wang, Heng Meng, Zekai Xiang, Jin Liu, Pingyi Zhou, Litao Chen, and Yongqiang Tang. 2026. Text2cad-bench: A benchmark for llm-based text-to-parametric cad generation. *arXiv preprint arXiv:2605.18430*.
- Ruiyu Wang, Yu Yuan, Shizhao Sun, and Jiang Bian. 2025a. Text-to-cad generation through infusing visual feedback in large language models. In *International Conference on Machine Learning*.
- Siyu Wang, Cailian Chen, Xinyi Le, Qimin Xu, Lei Xu, Yanzhou Zhang, and Jie Yang. 2025b. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 7880–7888.
- Karl DD Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G Lambourne, Armando Solar-Lezama, and Wojciech Matusik. 2021. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics (TOG)*, 40(4):1–24.
- Rundi Wu, Chang Xiao, and Changxi Zheng. 2021. Deepcad: A deep generative network for computer-aided design models. pages 6772–6782.
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muenighoff, Defu Lian, and Jian-Yun Nie. 2024. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*, pages 641–649.
- Haoyang Xie and Feng Ju. 2025. Text-to-cadquery: A new paradigm for cad generation with scalable large model capabilities. *arXiv preprint arXiv:2505.06507*.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Bo Yuan, Zelin Zhao, Petr Molodyk, Bin Hu, and Yongxin Chen. 2026. Clarify before you draw: Proactive agents for robust text-to-cad generation. *arXiv preprint arXiv:2602.03045*.

A Details of EXPCONCAD

A.1 Construction-aware Spatial Constraint Reasoning

In practice, the Construction-aware Spatial Constraint Reasoning consists of three stages. The first stage performs CAD construction structure understanding and generates queries for experience retrieval. The second stage conducts spatial constraint completion and produces an initial version of the CadQuery code. The final stage refines the generated code based on execution error messages.

CadQuery Function Guide Large language models commonly exhibit insufficient understanding of the CadQuery library, resulting in frequent syntax errors and semantic misuse of functions during code generation. To enhance model reliability, we introduce a curated *Function Guide* that organizes the usage patterns and implementation details of essential CadQuery APIs. The guide is integrated into all three stages of EXPCONCAD as supplementary knowledge, thereby improving generation accuracy and robustness. Figure 8 illustrates the constructed Function Guide.

Stage 1: Construction Structure Understanding and Query Generation The prompt used in Stage 1 is illustrated in Figure 9. To facilitate construction structure understanding, we categorize CAD modeling operations into four categories: *3D Primitive Generation*, *2D-Sketch & Extrude*, *Geometry Processing*, and *Spatial Transformation & Boolean Operation*. The first two categories are used to directly create CAD entities, the third category is used for geometric refinement operations such as filleting, and the last category is responsible for composing multiple entities.

During reasoning, the model first analyzes how the target geometry can be obtained through Boolean operations of several parts. Then, analyze how those parts should be constructed: via "3D Primitive Generation", "2D-Sketch & Extrude", or by further Boolean operations of sub-components. Based on this reasoning process, the model identifies the CAD elements, modeling operations, and constraint scopes from the shape description, thereby providing a structured foundation for the subsequent spatial constraint completion.

This stage also generates queries for experience retrieval. Based on the construction structure, the model generates fine-grained queries containing CAD elements and the modeling operations be-

tween them, facilitating the retrieval of relevant modeling experiences.

Stage 2: Spatial Constraint Completion and Code Generation

The prompt used in Stage 2 is illustrated in Figure 10. Prior to this stage, relevant experiences are retrieved using the query generated in Stage 1. We employ the BAAI General Embedding model (BGE), specifically bge-large-en-v1.5 (Xiao et al., 2024), to encode both the retrieval queries and the applicable constraint scope of the experiences into dense vector representations. Retrieval is then performed using cosine similarity in the embedding space, where the top- k nearest experiences are selected as auxiliary context. In our implementation, the top three retrieved experiences are injected into the model context.

Given the original shape description, the construction structure, and the retrieved experiences, the model infers the implicit constraints and performs numerical reasoning to derive geometric parameter values that satisfy these constraints. Based on the construction structure and derived parameters, the model subsequently generates the corresponding CadQuery code.

Stage 3: Code Refinement The prompt used in Stage 3 is illustrated in Figure 11. This stage focuses on correcting execution errors in the generated CadQuery code. When code execution fails, the executor returns the corresponding error messages to the model, which analyzes the failure causes and revises the implementation accordingly. The refinement stage only addresses syntax and implementation-level errors, without modifying the underlying modeling logic or numerical parameter configurations. The refinement process continues iteratively until the generated code executes successfully or a maximum of three refinement rounds is reached.

A.2 Experience memory

For each piece of data used for experience extraction, we first applied the construction-aware spatial constraint reasoning pipeline without experience enhancement to generate CadQuery programs. The programs were then executed and rendered into three-view images, which were subsequently evaluated by GPT-5 serving as the judge model. The judge assigned an initial score ranging from 0 to 10 and produced a corresponding diagnosis describing the major failure modes of the generated result. We

CadQuery Function Guide

1. **cq.Workplane('XY'|'YZ'|'ZX')**
 - * Action: Creates a workplane. This is the start of creating any solid.
 - * Note: Never use `cq.Workplane('XZ')`. Using 'ZX' ensures the local coordinates follow the Right-Hand Rule, where the local Y-axis (the normal) points in the global +Y direction.
2. **box(length, width, height)**
 - * Action: Generates a cuboid.
 - * Note: The **geometric center** is located at the workplane origin.
 - * X Range: `[-length/2, length/2]`, Y Range: `[-width/2, width/2]`, Z Range: `[-height/2, height/2]`.
 - * You must use `cq.Workplane(...).box(...)`; the box function cannot be used directly.
3. **cylinder(height, radius)**
 - * Action: Generates a cylinder aligned along the Z-axis.
 - * Note: The **geometric center** is located at the workplane origin.
 - * Z Range: `[-height/2, height/2]`. The bottom face is at `z = -height/2`, and the top face is at `z = +height/2`.
 - * You must use `cq.Workplane(...).cylinder(...)`; the cylinder function cannot be used directly.
4. **rect(xLen, yLen)**
 - * Action: Draws a rectangle.
5. **circle(radius)**
 - * Action: Draws a circle.
6. **polygon(nSides, diameter)**
 - * Action: Draws a regular N-sided polygon. 'nSides' is the number of sides, 'diameter' is the **diameter** of the circumscribed circle.
7. **moveTo(x, y) / lineTo(x, y)**
 - * Action: Moves the cursor or draws a straight line to coordinates (x, y).
8. **polyline([(x, y)])**
 - * Action: Connects a series of coordinates to draw a polyline.
9. **threePointArc((x1, y1), (x2, y2))**
 - * Action: Draws an arc starting from the current point, passing through the midpoint `(x1, y1)`, and ending at `(x2, y2)`.
10. **close()**
 - * Action: Closes the current path drawn by 'lineTo', 'polyline', or 'threePointArc'.
11. **extrude(until, both)**
 - * Action: Extrudes a 2D profile along the normal direction (usually the Z-axis). The 2D profile must be a closed loop drawn directly by 'rect', 'circle', 'polygon', or lines closed by 'close()'.
 - * Note: Extrusion is **unidirectional** (unless 'both=True' is specified). Therefore, the **bottom face sits on the workplane**. This differs from 'box' and 'cylinder' which are centered.
 - * Coordinate Range: The extrusion spans from **0** to 'until' along the local normal axis. For example:
 - * On 'Workplane('XY')': Extrudes along the **Z-axis** (`[0, \text{until}]`).
 - * On 'Workplane('YZ')': Extrudes along the **X-axis** (`[0, \text{until}]`).
 - * On 'Workplane('ZX')': Extrudes along the **Y-axis** (`[0, \text{until}]`).
12. **translate((x, y, z))**
 - * Action: Moves the entire solid by the vector `(x, y, z)`.
13. **rotate((x1, y1, z1), (x2, y2, z2), angle)**
 - * Action: Rotates the entire solid by 'angle' degrees
 - * Rotation axis is defined by two points: From `(x1, y1, z1)` to `(x2, y2, z2)`
14. **union(toUnion) / cut(toCut) / intersect(toIntersect)**
 - * Action: Merges with / Subtracts / Intersects with other solid.
 - * Note: Operations can only be performed on **3D solids**.
15. **edges(selector).fillet(radius)**
 - * Action: Applies a **fillet / rounding** to selected edges
 - * Common selector string:
 - * `"|Z"`, `"|X"`, `"|Y"`: edges parallel to Z / X / Y

Figure 8: CadQuery function guide.

Prompt Used for Construction Structure Understanding and Query Generation

Role:

You are a CAD planning assistant proficient in CadQuery library.

You will be provided with:

* A textual modeling requirement

Your task is to produce:

- 1) A step-by-step modeling plan;
- 2) A set of queries for retrieving constraint-related experience.

Phase1: Step-by-step Planning

Each step must contain:

- Type: one of the 4 categories
- Description: what to do (must include function names you will use)
- Expected Outcome: a short description + assign a variable name (final variable must be `r`)

Categories:

***[3D Primitive Generation]**

* Logic: `cq.Workplane('XY')` -> directly generate primitive -> solid

* Available Functions: `box`, `cylinder`

* Suitable for constructing boxes and cylinders.

***[2D-Sketch & Extrude]**

* Logic: `cq.Workplane('XY')` -> draw a closed path -> `extrude()` -> solid

* Available Functions: `polygon`, `moveTo`, `lineTo`, `polyline`, `threePointArc`, `close`, `extrude`

* Suitable for constructing non-standard prisms, such as polygonal or trapezoidal prisms.

***[Geometry Processing]**

* Logic: apply edge filleting to an existing solid

* Available Functions: `edges`, `fillet`

***[Spatial Transformation & Boolean Operations]**

* Logic: `translate`/`rotate` -> `cut`/`union`/`intersect`

* Available Functions: `translate`, `rotate`, `cut`, `union`, `intersect`

* Suitable for combining parts into complex shapes.

Note:

- The general analytical approach is: First, consider how the target geometry can be obtained through Boolean operations of several parts. Then, analyze how those parts should be constructed: via [3D Primitive Generation], [2D-Sketch & Extrude], or by further Boolean operations of sub-components
- We do not perform Boolean operations on 2D objects; 2D objects are only used to be extruded into 3D solids. Boolean operations are applicable only to 3D objects.
- Avoid applying fillets to geometries after Boolean operations. Instead, apply filleting to individual components first (e.g., those from [2D-Sketch & Extrude] or [3D Primitive Generation]), and then perform the Boolean merge.
- If fillets are not requested, do not add them at all.
- When the modeling requirement mentioned 'intersect', it refers to the spatial interpenetration or overlapping of volumes (visual intersection), not a Boolean Intersect that discards outlying parts. Most cases actually require a Boolean Union to implement this.
- Do not involve specific parameter settings in the plan; only specify which functions to use.

Please first analyze and then plan. Wrap your plan in `</plan>`.

Phase2: Constraint Query Generation

A query should include both the geometric features and the functions used to model those features. A complex shape often consists of multiple features; for example, an L-shaped plate with two holes possesses two distinct geometric features: "two holes" and the "L-shape."

You should select the most critical geometric features and generate no more than four queries. Generally, you should focus on features that require Boolean operations to create.

Format Example

...(analyse)

<plan>

STEP_1

Type: 3D Primitive Generation

Description: Create the main base plate using function `box`.

Expected Outcome: A rectangular base, variable `base`.

STEP_2

...

</plan>

<query>

- To model "two holes": Directly generate two cylinders using the 'cylinder' function, apply the correct 'translate' operations to position them, and then use the 'cut' operation to subtract them and create the holes.

- ...

</query>

Function Guide

**Do not use any CadQuery functions other than those listed below. **

...

Figure 9: Prompt used for construction structure understanding and query generation.

Prompt Used for Spatial Constraint Completion and Code Generation

You are a CAD designer proficient in the **CadQuery** library.

You will be provided with:

- * A textual modeling requirement
- * A modeling plan
- * Some modeling experience.

Your task consists of two phases:

1. **Numerical Inference:** Perform reasoning based on the modeling requirement, modeling plan and experience.
2. **Code Generation:** Write the actual CadQuery code.

Phase 1: Numerical Inference(CRITICAL: REASONING MODE)

The modeling plan only outlines the steps; you need to determine the specific parameters.

When determining specific parameters, you must first identify the constraints that must be satisfied to model a particular shape. For example, when cutting two holes in a plate, these holes must be smaller than the plate in the S_x and S_y dimensions, but greater than or equal to the plate in the S_z dimension to ensure they cut through the plate; at the same time, the holes must not overlap with each other.

Modeling experience provides a reference for identifying and handling these constraints. You should first determine whether the experiences are useful and then reference them accordingly; irrelevant or incorrect experiences should not be referenced.

Instructions for Inference Style:

- * **Adopt a First-Person "Think Aloud" Style:** Use phrases like "Let's try...", "Wait, let me check...", "This implies...", "Hold on, that doesn't fit..."
- * **Simulate Trial and Error:** Intentionally propose a dimension, calculate its bounds, check it against constraints, and if it fails, try again until it meets all constraints.

Phase 2: Code Generation

- * Output a standalone Python script starting with `import cadquery as cq`.
- * Assign the final CAD object to a global variable named `r` (e.g., `r = result`).

Wrap the final code in a Python markdown code block.

Note:

- When the modeling requirement mentioned 'intersect', it refers to the spatial interpenetration or overlapping of volumes (visual intersection), not a Boolean Intersect that discards outlying parts.

Function Guide

- * **Do not use any CadQuery functions other than those listed below. Do not pass parameters other than those listed.**
- * **However, the use of auxiliary functions—such as sin and cos from the Python math module—is permitted.**

...

Figure 10: Prompt used for spatial constraint completion and code generation.

Prompt Used for Code Refinement

```
# ROLE
You are a senior Python developer and an expert in 3D parametric modeling using CadQuery. Your objective is to debug,
troubleshoot, and fix CadQuery scripts accurately.

# WORKFLOW
When provided with buggy code and/or an error message, follow these exact steps:
1. Analyze: Carefully review the code and the error message to identify the root cause of the failure.
2. Explain: Provide a brief, concise explanation of what went wrong and how you will fix it.
3. Fix: Provide the corrected version of the code. Strictly preserve the original geometric intent and logic; only change what
is necessary to make the code run successfully.

# CONSTRAINTS & FORMATTING
- Complete Code: Always output the ENTIRE corrected script. Never use placeholders like `...` or `# rest of the code`.
- Imports: The script must explicitly start with `import cadquery as cq`.
- Output Variable: The final CAD object MUST be assigned to a global variable exactly named `r` (e.g., `r = result` or `r =
model`).
- Markdown: Wrap the final corrected code inside a single Python markdown code block (``python ... ``).

#### Function Guide
...
```

Figure 11: Prompt used for code refinement.

only retained samples with initial scores lower than 7, which we regard as CAD generation failures, for subsequent experience distillation.

The experience distillation process for each sample is formulated as a multi-turn iterative procedure with verification, ensuring that the retained experiences are validated through measurable improvements in generation quality. Specifically, given the shape description, construction structure, previously generated code, associated score and diagnostic feedback, we first analyze the root causes of the current CAD generation failure and distill a set of reusable modeling experiences. The prompt used for experience distillation is illustrated in Figure 12.

The distilled experiences are then injected back into the generation pipeline as additional contextual guidance. Conditioned on the combination of shape description, construction structure, and experiences, the model regenerates new code. The generated code is subsequently executed and rendered into three-view images and re-evaluated by GPT-5 to obtain updated scores and diagnoses.

If the score of regenerated result achieves an improvement of more than two points over the initial score, the corresponding experiences are considered verified and are stored in the final experience repository. Otherwise, the newly generated code, updated score, and revised diagnostic feedback are fed back into the experience distillation module for another round of reflection and refinement. This

iterative process continues until either a sufficient score improvement is achieved or the maximum number of three retry iterations is reached.

B Case Studies of EXPCONCAD

Figure 3 presents qualitative comparisons on four representative hard cases. These examples cover common but challenging implicit constraints in vague-shape-level Text-to-CAD generation, including multi-face through holes, hollow structures, internal partitions, rounded corners, and repeated local cutouts.

In the first case, the input describes a mounting-block-like rectangular prism with two circular holes on the top face, a larger hole on the vertical face, and four smaller holes surrounding the larger one. This requires inferring multi-face through-hole constraints and symmetric placement relations. Most baselines fail to preserve the complete hole topology or the spatial arrangement of holes on different faces. Our method better recovers the intended multi-face structure and the relative placement among holes.

In the second case, the target shape is an elongated hollow rectangular prism with rounded corners and triangular cutouts on two sides. Existing methods often collapse the hollow structure, miss the side cutouts, or generate unrelated local geometry. Our method better preserves the hollow body and recovers the side cutout interactions.

In the third case, the shape contains two parallel

Prompt Used for Experience Distillation

You are an **Experience Summarizer**. Your task is to extract reusable modeling experiences from the execution trajectories of a "Text-to-CadQuery Agent."

You will receive the following inputs:

1. The user's text description of the target shape.
2. A modeling plan.
3. The CadQuery code generated by the Agent.
4. The reviewer's evaluation of whether the final rendered result meets the requirements.
5. Some reference patterns.
6. (In multi-turn dialogue) Feedback on whether previous experiences worked.

Experience

During the process of modeling with CadQuery, parameter settings must satisfy specific constraints. For example, when cutting two holes in a plate, these holes must be smaller than the plate in the \$x\$ and \$y\$ dimensions, but greater than or equal to the plate in the \$z\$ dimension to ensure they cut through the plate; at the same time, the holes must not overlap with each other.

The agent may have overlooked these constraints, leading to modeling results that do not meet expectations. You need to analyze the cause of the error and then provide modeling experiences.

The experience should include a pattern, content, and code_example:

pattern:

Describes the scenario where the experience applies; it should include both the geometric features and the functions used to model those features. Geometric features should be concise and essential, without unnecessary modifiers.

For example: "To model 'two holes': Directly generate two cylinders using the 'cylinder' function, apply the correct 'translate' operations to position them, and then use the 'cut' operation to subtract them and create the holes."

You will receive some reference patterns; the patterns you summarize should be similar to these reference patterns.

content:

Represents the specific details of the experience; it should provide a detailed explanation of the geometric constraints involved and the mathematical relationships that the parameters must satisfy.

code_example:

Provides a schematic code snippet for illustration.

Multi-turn Dialogue

In subsequent rounds, you will receive feedback including:

- The updated code
- The reviewer's new evaluation.

Based on this feedback, continue adjusting your approach to resolve the issues raised in the feedback.

Note

- All experiences should focus on the setting of parameter values; do not change the functions used.
- The reviewer's evaluation is based directly on the final rendered result; therefore, when analyzing the cause of an error, you should consider how the final rendered output will actually look given the current parameter settings. For example, if a cutter is entirely contained within the base geometry, the external appearance will remain unchanged.
- Multiple experiences can be summarized from a single data point. The experiences must be reusable rules capable of guiding the solution for a whole class of problems.

Output Format

Perform the analysis first, then output in JSON format as follows:

```
...(analysis)
```json
[
 {
 "type": "spatial",
 "pattern": "...",
 "content": "...",
 "code_example": "..."
 }
]
```

```
...
]
```

#### ### Function Guide

**May help you analyse the cause of failure.**

...

Figure 12: Prompt used for experience distillation.

internal horizontal bars that create two rectangular voids. This requires inferring internal partition constraints and symmetry from a vague description. Baselines tend to generate a solid block, miss the internal bars, or produce incorrect separations. Our method better captures the parallel bars and the resulting void structure.

In the fourth case, the input describes a tall hollow rectangular prism with open top and bottom, rounded corners, and identical triangular cuts near the top and bottom. This case requires combining hollow-shell constraints, rounded-corner geometry, and repeated cutout constraints. Our method better preserves these coupled construction constraints, producing a CAD model closer to the ground truth.

Overall, the qualitative results show that EXP-CONCAD is especially effective when the intended shape depends on implicit spatial constraints rather than explicit dimensions. Together with the quantitative results, these cases demonstrate that CAD-context grounding, implicit constraint completion, and reusable experience jointly improve vague-shape-level Text-to-CAD generation.

To provide a more concrete understanding of how EXPCONCAD performs implicit spatial constraint completion, we present the full generation process for the third case, as illustrated in Figure 13. Faced with the vague shape input “The 3D shape is a rectangular prism with two parallel internal horizontal bars, creating two rectangular voids,” the model first performs construction structure understanding, identifying the interactions between the CAD elements “voids\_union” and “main\_body.” This establishes a solid structural basis for subsequent spatial constraint completion.

A query is then constructed and relevant experiences are retrieved. These experiences provide additional guidance on the constraints that should be considered for similar geometric patterns. For example, the “Through-Hole Constraint” is directly applicable to the current modeling task. During the implicit constraint completion process, the model leverages the construction structure together with the retrieved experiences to identify a set of constraints, such as ensuring that the voids are fully contained within the main body along the X and Y directions and that they cut through the main body. Based on these constraints, the model subsequently infers parameter configurations that satisfy all constraints.

Compared with other methods, our approach explicitly establishes interactions between CAD ele-

ments through construction structure understanding, making it easier to determine which inter-element constraints should be inferred. Furthermore, the introduction of the experience mechanism helps the model recognize additional necessary constraints. As shown in Figure 3, “Ours w/o EXP” ignores the constraint that the voids must be at least as large as the main body along a certain dimension in order to cut through it, resulting in voids that are entirely enclosed within the main body. By incorporating experiences, the model becomes aware of this constraint and successfully generates the correct structure.

## C Implementation Details

### C.1 Benchmark Information

We describe the benchmarks used in our experiments, including Text2CAD, CADFusion, and our constructed CADFusion-Hard. We report the dataset statistics and task settings for each benchmark, and further detail the construction procedure of CADFusion-Hard.

**Text2CAD.** Text2CAD (Khan et al., 2024) is built upon the DeepCAD (Wu et al., 2021) dataset and augments CAD models with multi-level natural language annotations generated by LLMs and VLMs. The benchmark defines four prompt levels, ranging from vague-shape-level descriptions (L0) to modeling-step-level descriptions (L3). CAD models are represented as sketch-and-extrude token sequences (Willis et al., 2021). In total, the benchmark contains approximately 600K training pairs and 32K validation/test pairs derived from around 150K CAD models.

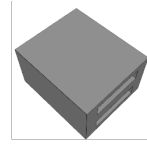
**CADFusion.** CADFusion (Wang et al., 2025a) is also based on DeepCAD and adopts sketch-and-extrude CAD representations. Unlike most Text-to-CAD datasets that emphasize modeling-step-level descriptions, CADFusion mainly focuses on vague-shape-level descriptions. The captions are first generated from rendered CAD images using VLMs, and are subsequently refined and verified by human annotators to improve annotation quality. The resulting benchmark contains approximately 20K text-CAD pairs, among which 952 samples are used for testing.

### Implicit-constraint hard subset construction.

To evaluate models on genuinely challenging shape descriptions with implicit spatial constraints, we

### Vague Shape Input

The 3D shape is a rectangular prism with two parallel internal horizontal bars, creating two rectangular voids.



### Construction Structure Understanding

```
...
STEP_3
Type: Spatial Transformation & Boolean Operations
Description: Create the second void bar by duplicating the logic (or creating a second instance) and applying `translate` to position it symmetrically opposite to the first bar relative to the center. Then, use `union` to combine the two void bars into a single solid.
Expected Outcome: A combined solid containing both void shapes in their correct positions, variable `voids_union`.
STEP_4
Type: Spatial Transformation & Boolean Operations
Description: Apply the `cut` operation to subtract the `voids_union` from the `main_body`.
Expected Outcome: The final rectangular prism with two internal rectangular voids, variable `r`.
```

### Retrieval Query

- To model "two parallel internal horizontal bars creating voids": Generate two rectangular prisms using the `box` function to represent the voids, apply `translate` to ...

### Retrieval Experience

Pattern: To model a 'hollow rectangular prism' (tube/beam): Generate an outer `box` and an inner `box` representing the void...

Experience: When creating a hollow structure by subtracting an inner solid from an outer solid, the dimensions of the inner solid must satisfy specific constraints relative to the outer solid.

1. **Through-Hole Constraint**: If the target is a tube or beam with open ends, the dimension of the inner solid along the longitudinal axis (the axis of the beam) must be greater than or equal to the corresponding dimension of the outer solid. If the inner solid is shorter, the result will have solid end caps, failing to represent a hollow tube.
2. **Wall Thickness Constraint**: The dimensions of the inner solid in the cross-sectional directions (width and height) must be strictly smaller than the outer solid. The difference determines the wall thickness.
3. **Centering**: Both solids must be centered at the same origin to ensure uniform wall thickness.

Code Example: `outer_box = cq.Workplane('XY').box(length=300, width=50, height=30)`  
# Inner box length must be  $\geq$  outer length to cut through  
`inner_box = cq.Workplane('XY').box(length=300, width=40, height=20)`  
`r = outer_box.cut(inner_box)`

### Spatial Constraint Completion

```
...
* "Internal": They must be fully contained within the main body in the X and Y directions, but to create a "void" (a hole), they must cut through the main body.
* Constraint Check (Experience Reference): The provided experiences emphasize that to create a *through-hole* (a visible void), the dimension of the cutter along the axis of the cut must be greater than or equal to the dimension of the base.
 * If the bars run along the X-axis, the "cut" happens along the X-axis.
 * Therefore, the length of the void bar (SL_{void}) must be $\geq L_{\text{main}}$ (100). Let's set $SL_{\text{void}} = 100$ to ensure it cuts all the way through.
 * The width (SW_{void}) and height (SH_{void}) must be strictly smaller than the main body's width and height to leave material (walls).
...
* Let's refine the dimensions.
 * Main: $100 \times 60 \times 40$.
 * Void Bar: Length 100 (to cut through X), Width 20, Height 10.
 * Position 1: $(0, 0, 10)$.
 * Position 2: $(0, 0, -10)$.
 * Check overlap: Bar1 is $z \in [5, 15]$, Bar2 is $z \in [-15, -5]$. No overlap.
 * Check containment:
 * X: $[-50, 50]$ (matches main).
 * Y: $[-10, 10]$ (inside $[-30, 30]$).
 * Z: Inside $[-20, 20]$.
 * This works.
...
```

Figure 13: The full generation process for the third case in Figure 3.

construct hard subsets from both CADFusion and Text2CAD. For each benchmark, we first run a pool of standard baseline methods, including vanilla prompting and CoT prompting, for three independent runs, and compute the average VLM score for each sample across all method-run combinations. We use this average score as an automatic difficulty proxy and select the 500 samples with the lowest scores as candidate hard cases. However, low VLM scores may also result from noisy descriptions, low-quality samples, or ambiguous ground truth rather than genuine spatial reasoning difficulty. Therefore, we further conduct human difficulty assessment. Three CAD-experienced annotators independently rate each candidate from 0 to 10, producing a human difficulty score by averaging their ratings. Annotators are instructed to assign high scores to valid but underspecified shape descriptions whose difficulty mainly comes from missing implicit spatial constraints, such as constraint scopes, feature placements, alignments, containment, spacing, or cut-through relations, and to assign low scores to noisy, unclear, or low-quality samples. We then select the 200 samples with the highest human difficulty scores from each benchmark to form CADFusion-Hard and Text2CAD-Hard, respectively. This procedure filters out low-quality cases while retaining samples whose difficulty mainly comes from missing spatial constraints.

## C.2 Experience Construction

In the practical implementation of the experience distillation process, We randomly sampled 1,500 instances from the CADFusion (Wang et al., 2025a) training set for experience construction.

## C.3 Evaluation Metrics

We evaluate generated outputs from three complementary perspectives: (i) text–shape consistency, (ii) geometric fidelity to the reference shape, and (iii) executability of the generated CADQUERY program.

**VLM score.** We use VLM score to evaluate the semantic consistency between the input description and the generated CAD model.

Traditional VLM-based evaluation typically renders a 3D shape into orthographic three-view images before scoring, referred to as  $\mathbf{VLM}_{3v}$ . However, the visual information provided by three-view renderings is inherently limited. We pro-

pose  $\mathbf{VLM}_{2v}$ . Given a generated shape, we render it from two informative viewpoints and query a vision-language model to assess whether the rendered object matches the input text description. Let  $x$  denote the text prompt,  $I^{(1)}$  and  $I^{(2)}$  the two rendered views, and  $f_{\text{vlm}}(x, I^{(1)}, I^{(2)})$  the scalar score produced by the VLM. The metric is defined as

$$\mathbf{VLM}_{2v}(x, \hat{S}) = f_{\text{vlm}}(x, I^{(1)}(\hat{S}), I^{(2)}(\hat{S})), \quad (10)$$

where  $\hat{S}$  denotes the generated 3D shape.

Unlike conventional orthographic three-view renderings (*i.e.*, front, top, and right), we adopt two complementary diagonal perspective views. This design is motivated by viewpoint selection principles in computer vision and graphics, where informative viewpoints are expected to maximize visible surface information while minimizing occlusion redundancy (Vázquez et al., 2001). Although orthographic projections are well suited for precise engineering specification, they are often sub-optimal for perceptual completeness and semantic understanding. In particular, axis-aligned views frequently suffer from severe self-occlusion, causing substantial surface regions to become hidden or collapse into edge-aligned projections. By contrast, diagonal viewpoints reveal multiple principal faces simultaneously, substantially increasing effective surface visibility compared to standard three-view renderings, as shown in Figure 14, where features on the bottom face—largely obscured in the three-view renderings—become clearly visible, highlighting the advantage of the two-view perspective in conveying complete surface information.

Following this principle, each mesh is first normalized into a unit bounding box and centered at the origin. We then place two virtual cameras along approximately opposite diagonal directions relative to the object center, corresponding to the positive and negative octants: (1,1,1) and (-1,-1,-1). Both cameras are oriented toward the object centroid, while their focal distances are adaptively adjusted according to object scale to ensure complete coverage. This configuration produces two complementary isometric-like views that jointly capture the front/back, left/right, and top/bottom geometric structures.

The exact evaluation prompt used for VLM-based scoring is shown in Figure 15. Following CADFusion (Wang et al., 2025a), the prompt is designed around three complementary evalua-

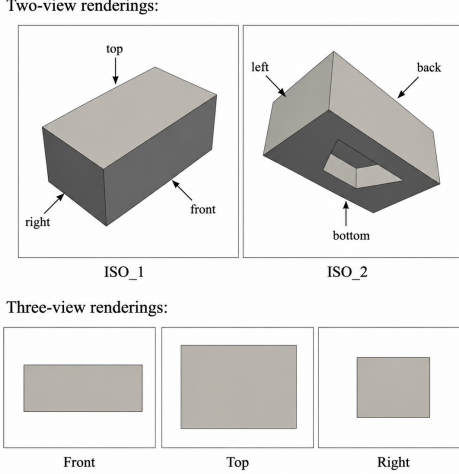


Figure 14: Comparison of Two-view and Three-view Renderings.

tion dimensions: (1) *global topology*, which measures whether the overall geometric structure aligns with the textual description; (2) *component fidelity and count*, which verifies the presence, geometric correctness, and exact number of key components (e.g., holes and protrusions); and (3) *spatial distribution*, which evaluates whether the relative scale and placement of components are correct, as well as whether abnormal intersections or other structural inconsistencies are present. Together, these dimensions encourage the VLM to perform holistic geometric reasoning rather than relying on superficial visual cues. To further mitigate rendering-induced bias, the prompt explicitly instructs the VLM to ignore color, texture, and other non-geometric cues, focusing solely on structural and spatial correctness. The VLM is required to return both a textual rationale and a single integer score ranging from 0 to 10, where higher scores indicate stronger text–shape consistency.

To further evaluate the alignment between our evaluation metric and human preference, we conduct a human correlation study. We randomly sample 50 text–shape pairs and recruit five volunteer annotators with proficient CAD modeling experience to independently assess text–shape consistency on a 10-point scale. During evaluation, annotators are allowed to freely inspect each 3D model from arbitrary viewpoints using interactive visualization software. To ensure consistency between human assessment and the automatic evaluation framework, annotators are instructed to score each sample according to the same three evaluation dimensions used in the VLM prompt (Figure 15). The final

Method	Spearman $\rho$
$\mathbf{VLM}_{3v}$	0.71
$\mathbf{VLM}_{2v}$	<b>0.84</b>

Table 3: Spearman correlation between automatic metrics and human judgments on text–shape consistency. Higher values indicate better alignment with human preference.

human score for each sample is obtained by averaging the ratings across annotators. We then compute the Spearman rank correlation coefficient to measure the ranking consistency between human judgments and two automatic metrics, namely  $\mathbf{VLM}_{2v}$  and  $\mathbf{VLM}_{3v}$ . Given paired samples  $\{(y_i, \hat{y}_i)\}_{i=1}^N$ , where  $y_i$  and  $\hat{y}_i$  denote human and model scores, respectively, the Spearman correlation is defined as

$$\rho = 1 - \frac{6 \sum_{i=1}^N d_i^2}{N(N^2 - 1)}, \quad (11)$$

where  $d_i$  is the difference between the ranks of  $y_i$  and  $\hat{y}_i$ .

The results, shown in Table 3, indicate that  $\mathbf{VLM}_{2v}$  achieves higher correlation with human judgments than  $\mathbf{VLM}_{3v}$ , suggesting that the proposed two-view rendering strategy better aligns with human perceptual preferences for text–shape consistency evaluation.

**Rotation Invariant IoU.** The Intersection-over-Union (IoU) is a common metric for evaluating the volumetric overlap between a predicted voxel grid  $V_{\text{pred}}$  and a ground-truth voxel grid  $V_{\text{gt}}$  (Kolodiazhnyi et al., 2025), defined as

$$\text{IoU} = \frac{|V_{\text{pred}} \cap V_{\text{gt}}|}{|V_{\text{pred}} \cup V_{\text{gt}}|}. \quad (12)$$

In our text-to-CAD generation task, vague-shape-level descriptions often specify only the rough geometry of an object without constraining its orientation. As a result, the predicted object may be rotated relative to the ground-truth canonical orientation, which can lead to an unfair evaluation: for example, a cuboid rotated by  $90^\circ$  around any principal axis remains semantically consistent with the text prompt, yet its voxel-wise overlap with the ground-truth in the fixed canonical orientation would be substantially reduced. To obtain an objective assessment that is insensitive to such pose variations, we adopt a rotation-invariant IoU (r-IoU) metric.

Specifically, we observe that in practice, most orientation misalignments occur as discrete rotations by multiples of  $90^\circ$  along the principal  $x, y,$

### Prompt Used for Scoring Text--Shape Consistency

You are a strict CAD Design Evaluator for novice designers.

Your task is to audit the geometric accuracy of a CAD model based on a textual description and a provided 2D rendered image.

#### ### IMAGE CONTEXT:

The provided image is a composite of two diagonal isometric views designed for full-surface coverage. You must synthesize information from both complementary perspectives to reconstruct the complete 3D geometry, including features that may be occluded in a single view.

#### ### EVALUATION CRITERIA:

Analyze the design based on the following three dimensions:

1. **Global Topology:**
  - Does the fundamental geometric form (e.g., boxy, L-shaped) align with the text description across all views?
2. **Component Fidelity & Count:**
  - **Completeness:** Are all described parts present?
  - **Shape Accuracy:** Is the shape of each individual component correct (e.g., is a cylinder actually cylindrical, not cubic)?
  - **Feature Verification:** Pay critical attention to specific features, especially the exact count of circular holes and protrusions.
3. **Spatial Distribution & Logic:**
  - **Relative Scale:** Are the proportions correct? (e.g., is Part A correctly sized relative to Part B according to the textual description?)
  - **Relative Positioning:** Are components placed in the correct locations relative to each other?
  - **Interference Check:** Detect any abnormal intersections. (e.g., two holes intersect)

#### ### IMPORTANT RULES:

- **STRICTLY IGNORE:** Color, material, textures, lighting, shadows, rendering style, and background.
- **FOCUS ONLY:** On structural accuracy, geometric logic, and spatial consistency.

#### ### SCORING SCALE:

- **0-3:** Major structural mismatch, missing core components, or severe layout errors.
- **4-7:** Correct overall shape but contains errors in component count, specific shapes, or spatial arrangement.
- **8-10:** Perfect or near-perfect geometric representation matching the description.

#### ### OUTPUT FORMAT:

You must respond with a JSON object containing:

- "reasoning": A concise analysis synthesizing findings from the three views (Max 3 sentences).
- "score": An integer from 0 to 10.

Do not include any other explanatory text.

Example:

```
```json
{
  "reasoning": "The overall L-shape is correct across all views. However, the Top View reveals that only 2 out of 4 required circular holes are present.",
  "score": 5
}
```
```

Figure 15: Prompt used for scoring text--shape consistency.

or  $z$  axes. To account for these discrepancies, we generate 24 discrete rotations of the ground-truth voxel grid for each object. These rotations correspond to the 24 elements of the cube rotation group (Conway et al., 2016). Since this group captures every possible combination of  $90^\circ$  rotations, it effectively covers all major canonical orientations a typical CAD object may assume. Given the predicted voxel grid  $V_{\text{pred}}$  and the set of rotated ground-truth voxel grids  $\{V_{\text{gt}}^{(r)}\}_{r=1}^{24}$ , the rotation-invariant IoU is defined as:

$$\text{r-IoU} = \max_{r \in \{1, \dots, 24\}} \frac{|V_{\text{pred}} \cap V_{\text{gt}}^{(r)}|}{|V_{\text{pred}} \cup V_{\text{gt}}^{(r)}|}. \quad (13)$$

This approach ensures that evaluation focuses on shape correctness rather than penalizing orientation misalignment.

**Chamfer Distance.** The Chamfer Distance (CD) is another commonly used metric for evaluating geometric similarity between 3D shapes (Khan et al., 2024; Kolodiazhnyi et al., 2025; Wang et al., 2025a). It measures geometric similarity between two point sets sampled from mesh surfaces. Given a predicted point cloud  $P$  and a ground-truth point cloud  $G$ , CD is defined as

$$\begin{aligned} \text{CD}(P, G) = & \frac{1}{|P|} \sum_{p \in P} \min_{g \in G} \|p - g\|_2^2 \\ & + \frac{1}{|G|} \sum_{g \in G} \min_{p \in P} \|g - p\|_2^2. \end{aligned} \quad (14)$$

This metric captures how closely the predicted surface approximates the ground-truth geometry. We uniformly sample 2000 points from each mesh surface for CD computation. Following common practice (Khan et al., 2024), all reported CD values in the paper are multiplied by 1000 for readability.

**Rotation Invariant Chamfer Distance.** Similar to rotation-invariant IoU, we adopt the Rotation Invariant Chamfer Distance (r-CD) to account for unknown orientation. Specifically, we evaluate the Chamfer Distance across a discrete set of 24 pre-rotated ground-truth point clouds  $\{G^{(r)}\}_{r=1}^{24}$  and take the minimum:

$$\text{rCD} = \min_{r \in \{1, \dots, 24\}} \text{CD}(P, G^{(r)}) \quad (15)$$

**Invalid ratio.** A generated program is useful only if it can be successfully executed to produce a valid 3D model. Programs may fail due to syntax or

runtime errors; let  $N_{\text{total}}$  denote the total number of generated programs and  $N_{\text{invalid}}$  the number of failures. The invalid ratio is defined as

$$\text{Inv.} = \frac{N_{\text{invalid}}}{N_{\text{total}}}. \quad (16)$$

A lower invalid ratio indicates more reliable program generation.

For VLM evaluation, we report the average score across the test set. For r-IoU, CD, and r-CD, we report both the mean and median values. The **mean** captures the overall expected performance and is sensitive to outliers, reflecting the aggregate behavior of the model. The **median** is more robust to extreme cases and better characterizes the typical per-instance performance.

To avoid overestimating methods with high evaluation failure rates, we adopt **validity-aware evaluation** in Table 2. Metrics are computed over the entire test set rather than conditioning only on successfully evaluated samples, which would otherwise bias results toward methods that fail more frequently but perform well on a small subset of valid outputs. For samples without valid outputs, we apply a conservative imputation strategy using tail quantiles estimated from valid samples: the 90th percentile for error metrics (CD and r-CD, where larger values indicate worse performance) and the 10th percentile for reward-like metrics (r-IoU and VLM score, where smaller values indicate worse performance). This approximates near-worst observed behavior while avoiding instability from extreme outliers, yielding a more faithful estimate of expected model performance.

#### C.4 Baseline Reproduction Details

For Text2CAD (Khan et al., 2024), Cadrille (Kolodiazhnyi et al., 2025), CAD-Coder (Guan et al., 2026), and CadFusion (Wang et al., 2025a), we strictly use their publicly released code and pretrained model weights. For CADCodeVerify (Alrashedy et al., 2025), due to the absence of official open-source code, we re-implemented the method based on the descriptions and prompts provided in the original paper.

For Vanilla, CoT, and our proposed framework across different backbone models, we set the temperature to 0 and max\_tokens to 10000. For Qwen3.5-27B, the “thinking mode” is disabled, while for GPT-5, the reasoning\_effort is set to minimal. All other parameters remain at their default

settings.

During VLM evaluation, we use GPT-5 with a temperature of 0, max\_tokens set to 4,096, and reasoning\_effort set to minimal. All remaining parameters are kept at their default values.

All of our experiments are conducted on A100 GPUs, and the closed-source models are accessed through official channels.

### C.5 Additional Settings for Diagnostic Experiments

This section provides detailed settings for the diagnostic experiments in Section 4.3 and Section 4.4. These experiments are designed to analyze the contribution of individual components in EXPCONCAD, rather than to provide another full benchmark comparison against prior methods.

For the main comparison in Section 4.2, we follow the penalized evaluation protocol described in Section 4.1, where non-renderable outputs are included in the reported metrics. For the diagnostic experiments, we compute VLM on successfully rendered outputs. For the ablation study, we additionally report IR to show how verification and other components affect renderability. For the two design analyses in Section 4.4, the compared variants have similar renderability, with IR differences within 3 percentage points. Therefore, we report VLM as the primary diagnostic metric to focus on text-shape consistency.

**Settings for the Ablation Study** This setting corresponds to the ablation study in Section 4.3. We evaluate all variants on CADFUSION-HARD, using the same evaluation split as in the main experiments. We use Qwen3.5-27B and GPT-5 as backbones, following the same generation settings as the main comparison.

The ablation progressively adds the major components of EXPCONCAD. *Prompt* denotes a strong task-specific prompting baseline. *+Verif.* adds execution-based verification and revision to reduce non-renderable outputs. *+CSU* further introduces CAD Construction Structure Understanding, which identifies the relevant CAD elements, operations, and constraint scopes. *+SCC* adds Spatial Constraint Completion over the recovered construction structure. *Ours* further incorporates experience-enhanced SCC.

For this diagnostic experiment, we report VLM on successfully rendered outputs and IR separately. This allows us to analyze both text-shape consistency

and renderability when progressively adding framework components. The numerical results are reported in Table 7.

**Settings for Analyzing Construction Structure Understanding** This setting corresponds to the first design analysis in Section 4.4, which studies whether Construction Structure Understanding provides a better basis for Spatial Constraint Completion. We conduct this experiment on CADFUSION-HARD with the same evaluation split used in the main experiments. We evaluate both Qwen3.5-27B and GPT-5.

We compare three settings. The first is a verification-only baseline, which uses execution-based verification and revision but does not explicitly perform Construction Structure Understanding or Spatial Constraint Completion. The second performs direct Spatial Constraint Completion from the underspecified shape description, without first recovering the CAD construction structure. The third follows the proposed design, where the model first performs Construction Structure Understanding and then completes spatial constraints over the recovered constraint scopes.

This experiment isolates whether SCC should be performed directly from the shape-level description or based on the recovered construction structure. Since the compared variants have similar renderability, with IR differences within 3 percentage points, we report VLM as the primary metric.

**Settings for Analyzing Experience Memory** This setting corresponds to the second design analysis in Section 4.4, which studies how the size and quality of experience memory affect Spatial Constraint Completion. We conduct this experiment on CADFUSION-HARD with the same evaluation split used in the main experiments. We evaluate Qwen3.5-27B and GPT-5 as backbones.

For each backbone, we keep the overall EXPCONCAD pipeline fixed and vary only the size of the experience memory. The experience memory is constructed from prior CAD construction cases, and the number of stored experience items follows the settings shown in Figure 6. For Qwen3.5-27B, the compact memory contains 50 experience items constructed from 45 source cases. For GPT-5, the largest memory contains 500 experience items constructed from 158 source cases. During inference, retrieved experiences are used only to guide Spatial Constraint Completion, while the benchmark evaluation split remains unchanged.

This experiment analyzes whether additional experience improves SCC and whether the benefit comes from memory size alone or from the granularity and relevance of the stored experience. Since IR remains comparable across memory-size settings, with differences within 3 percentage points, we report VLM as the primary metric.

**Settings for Cross-Dataset Generalization Analysis** This setting corresponds to the generalization analysis in Section 4.5. To study cross-dataset transfer, we conduct an additional experiment with Qwen3.5-27B on two hard sets, CADFUSION-HARD and TEXT2CAD-HARD. CADFUSION-HARD is closer to the source distribution used by CADFUSION and by our experience construction, while TEXT2CAD-HARD serves as an unseen cross-dataset test set.

CADFUSION learns an end-to-end mapping from CADFUSION-side samples. In contrast, our framework explicitly completes implicit spatial constraints by first understanding the CAD construction structure, and further enhances Spatial Constraint Completion with an experience memory constructed only from CADFUSION-side source cases. Therefore, this experiment evaluates whether end-to-end learned generation patterns, explicit construction-aware constraint completion, and retrieved constraint-completion experiences can transfer to unseen shape-description distributions. We report VLM as the main metric in Section 4.5, and provide full results on Qwen3.5-27B in Appendix D.3.

## D Supplementary Experimental Analysis

### D.1 Results under the Conventional Valid-Output Protocol

We additionally report the original evaluation results under the conventional Text-to-CAD evaluation protocol in Table 4. Following prior work, VLM, r-IoU, CD, and rCD are computed only on successfully rendered outputs, while Invalid Ratio is reported separately. Therefore, these metrics reflect the quality of valid generations, but do not directly penalize methods for failed rendering cases. This protocol is useful for comparing the geometric and text-shape quality of renderable outputs, whereas the validity-aware results in the main paper evaluate the full generation pipeline by treating invalid outputs as task failures.

### D.2 Results on the Full CADFUSION

Table 5 shows that our two variants achieve the best VLM scores on the full CADFUSION benchmark. This confirms the effectiveness of the CG-ICC pipeline on the general test set. However, compared with CADFUSION-HARD, experience brings less stable gains. The reason is that the full benchmark contains many easier cases whose missing constraints can already be completed by the structured pipeline. Thus, experience improves Qwen3.5-27B from 7.56 to 7.74, but slightly hurts GPT-5, where the stronger base model may not need additional experience guidance for these simpler cases.

### D.3 Results on the TEXT2CAD-HARD

See in Table 6.

### D.4 Ablation Study

Table 7 shows that EXPCONCAD consistently achieves the best VLM score on both Qwen3.5-27B and GPT-5. Compared with Vanilla and CAD Prompt, the full framework brings clear improvements, indicating that a structured generation pipeline is more effective than prompt design alone. Removing CG or ICC weakens performance, showing that both CAD-context grounding and implicit constraint completion are useful for vague-shape-level Text-to-CAD. Moreover, the gap between **w/o Exp.** and **Ours** shows that reusable experience further improves the quality of constraint completion.

### D.5 Additional Analysis of Experience in EXPCONCAD

This section provides additional analysis of the experience mechanism in EXPCONCAD. We show how different backbones produce experiences at different granularities, and how this affects the usability and retrieval effectiveness of the experiences.

Figure 16 illustrates the differences in experiences summarized by Qwen3.5-27B and GPT-5 for the same source case. The source case is described as: "The 3D shape is a vertical cylinder with a height significantly greater than its diameter. At its center, there is a cylindrical hole containing a small solid cylinder." It involves two distinct patterns: (1) constructing a cylindrical hole; (2) modeling a small cylinder inside the hole.

Qwen3.5-27B summarizes a single experience whose pattern simultaneously covers both constructing the hole and inserting the cylinder. However, its content only specifies the constraint for

| Method                            | CadFusion-Hard |                     |                      |                    |                     |                     |                      |                   |
|-----------------------------------|----------------|---------------------|----------------------|--------------------|---------------------|---------------------|----------------------|-------------------|
|                                   | VLM $\uparrow$ | m. r-IoU $\uparrow$ | md. r-IoU $\uparrow$ | m. CD $\downarrow$ | md. CD $\downarrow$ | m. rCD $\downarrow$ | md. rCD $\downarrow$ | Inv. $\downarrow$ |
| <i>Trained Text-to-CAD Models</i> |                |                     |                      |                    |                     |                     |                      |                   |
| Text2CAD                          | 1.88           | 0.182               | 0.140                | 104.983            | 90.268              | 70.634              | 51.074               | <b>0.00</b>       |
| Cadrille                          | 1.18           | 0.043               | 0.022                | 151.521            | 138.824             | 79.800              | 70.911               | 5.00              |
| CAD-Coder                         | 1.57           | 0.290               | 0.263                | 65.237             | 49.155              | 37.554              | 26.538               | 3.00              |
| CadFusion                         | 4.90           | 0.343               | 0.331                | 52.149             | 42.988              | 24.631              | 17.924               | 23.50             |
| <i>Qwen3.5-27B</i>                |                |                     |                      |                    |                     |                     |                      |                   |
| Vanilla                           | 4.68           | <u>0.356</u>        | 0.327                | 52.358             | 36.661              | 23.958              | <b>13.966</b>        | 50.50             |
| CoT                               | 5.18           | 0.353               | 0.283                | 52.213             | <b>31.758</b>       | <u>21.981</u>       | 16.007               | 67.00             |
| CADCodeVerify                     | 5.60           | <b>0.357</b>        | <b>0.343</b>         | <b>49.321</b>      | 32.743              | 22.625              | 16.397               | 13.50             |
| <b>Ours w/o Exp.</b>              | <u>5.67</u>    | 0.345               | <u>0.336</u>         | <u>49.682</u>      | <u>32.589</u>       | 22.192              | 14.201               | <u>1.00</u>       |
| <b>Ours</b>                       | <b>6.34</b>    | 0.347               | 0.325                | 50.672             | 34.109              | <b>21.388</b>       | <u>14.193</u>        | 2.00              |
| <i>GPT-5</i>                      |                |                     |                      |                    |                     |                     |                      |                   |
| Vanilla                           | 5.89           | 0.372               | 0.344                | 54.463             | 31.330              | 21.525              | 13.202               | 38.50             |
| CoT                               | 5.48           | 0.360               | 0.327                | 54.782             | 38.583              | 21.061              | 14.883               | 32.00             |
| CADCodeVerify                     | 6.00           | <b>0.378</b>        | <b>0.372</b>         | <u>49.202</u>      | <u>29.224</u>       | 21.357              | <u>12.964</u>        | 21.00             |
| <b>Ours w/o Exp.</b>              | <u>6.16</u>    | 0.362               | <u>0.353</u>         | <u>49.954</u>      | <u>32.062</u>       | <u>20.330</u>       | 13.239               | <u>0.50</u>       |
| <b>Ours</b>                       | <b>6.62</b>    | <u>0.373</u>        | 0.352                | <b>47.157</b>      | <b>28.680</b>       | <b>19.307</b>       | <b>11.032</b>        | <u>0.50</u>       |

Table 4: Main results on CADFUSION-HARD. Best and second-best results are shown in **bold** and underlined.

inserting the cylinder, namely that the cylinder’s radius should be smaller than the hole’s radius. Consequently, even when this experience is retrieved, it cannot provide guidance for constructing the hole, limiting its usability for tasks that require hole creation.

In contrast, GPT-5 generates two separate experiences, each aligned with a specific pattern. The first experience focuses on constructing the hole, specifying in its content that the hole height must exceed the body height. The second experience focuses on inserting the cylinder, explicitly detailing the corresponding constraints. This separation allows GPT-5 to provide targeted guidance for each pattern.

As a result, GPT-5 offers more generalizable and reusable experiences across different CAD contexts. Its fine-grained, pattern-specific experiences improve retrieval accuracy and task performance, demonstrating greater versatility compared to the coarse, case-level guidance produced by Qwen3.5-27B.

## D.6 Token Consumption Analysis

We report the token consumption of different methods in Figure 17. Although our method achieves strong performance, this improvement comes at the cost of increased token usage, and the overhead

varies across backbone models.

When using Qwen3.5-27B as the backbone, we observe that the model occasionally falls into repetitive generation loops, repeatedly producing duplicated content until the context window is exhausted. This behavior appears to be associated with limitations in the model’s reasoning capability and is more likely to occur on challenging cases requiring complex reasoning. As a result, the average token consumption is significantly inflated. Specifically, under the Qwen3.5-27B backbone, repetitive generation occurs in 8.5% of the samples for *Ours w/o Exp* and 10% for *Ours*.

In contrast, such context-saturating repetition is absent when GPT-5 is adopted as the backbone. Moreover, under the GPT-5 backbone, our framework not only outperforms CADCodeVerify in task performance but also achieves lower token consumption.

## Source Case

The 3D shape is a vertical cylinder with a height significantly greater than its diameter. At its center, there is a **cylindrical hole** containing a **small solid cylinder**.



### Exp 1



"pattern": "To model 'a hole containing a small solid cylinder': **Generate a cylinder for the hole using the 'cylinder' function, apply 'cut' to create a void in the main body, then generate a smaller cylinder for the inner solid and use 'union' to merge it into the void.**"

"content": "When modeling a cavity (hole) that contains an inner object, **the geometric constraint requires that the radius of the inner solid cylinder be strictly less than the radius of the hole cylinder.** If the inner cylinder's radius is equal to the hole's radius, the 'union' operation will completely fill the void, mathematically reconstructing the original solid volume. This results in a featureless solid cylinder where the 'hole' and the distinct 'inner cylinder' are not visible. To preserve the visual appearance of a hole containing an object, the inner cylinder must be smaller, leaving a visible gap."

### Exp 1



"pattern": "To ensure 'a through cylindrical hole' in a centered cylinder created by 'cylinder': **Create the hole as a 'cylinder' aligned coaxially and use 'cut' to subtract;** set the hole cylinder height greater than or equal to the body height."

"content": "Because 'cylinder(height, radius)' is centered with  $z \in [-\text{height}/2, +\text{height}/2]$ , a through-hole requires the cutter to span at least the body's Z range. **Thus, enforce hole\_height  $\geq$  body\_height.** If a Z offset  $\Delta z$  is applied to the hole cutter, ensure it still covers the body in Z:  $\text{hole\_height} \geq \text{body\_height} + 2*|\Delta z|$ , or equivalently, the cutter's bottom  $z \leq -\text{body\_height}/2$  and top  $z \geq +\text{body\_height}/2$ ."

### Exp 2

"pattern": "**To model 'a concentric inner pin inside a hole' using 'cylinder', 'translate', and 'union':** Generate the pin as a cylinder, ensure its radius is smaller than the hole's, keep it coaxial, and merge it with 'union'."

"content": "**Set pin\_radius  $<$  hole\_radius to preserve an annular air gap or material difference.** If the intent is a tight fit with no gap yet still distinguish topology, recognize that boolean-wise, if  $\text{pin\_radius} \geq \text{hole\_radius}$  and their axes coincide, the union after a prior cut will eliminate the hole entirely. For a visible annular region, enforce  $\text{pin\_radius} \leq \text{hole\_radius} - g_{\text{min}}$ , where  $g_{\text{min}}$  is the intended clearance. Heights should meet the positional intent:  $\text{pin\_height} \geq \text{body\_height}$  if the pin is through, otherwise define a shorter pin\_height and adjust its Z translation so it resides within the hole bounds."

Figure 16: Comparison of experiences generated by Qwen3.5-27B and GPT-5.

| Method                            | CadFusion      |                     |                      |                    |                     |                     |                      |                   |
|-----------------------------------|----------------|---------------------|----------------------|--------------------|---------------------|---------------------|----------------------|-------------------|
|                                   | VLM $\uparrow$ | m. r-IoU $\uparrow$ | md. r-IoU $\uparrow$ | m. CD $\downarrow$ | md. CD $\downarrow$ | m. rCD $\downarrow$ | md. rCD $\downarrow$ | Inv. $\downarrow$ |
| <i>Trained Text-to-CAD Models</i> |                |                     |                      |                    |                     |                     |                      |                   |
| Text2CAD                          | 2.25           | 0.210               | 0.150                | 93.000             | 83.910              | 59.860              | 43.950               | <b>0.00</b>       |
| Cadrille                          | 1.26           | 0.048               | 0.021                | 160.193            | 143.754             | 84.065              | 75.201               | 3.50              |
| CAD-Coder                         | 2.18           | 0.307               | 0.272                | 63.518             | 48.638              | 38.183              | 24.449               | 1.89              |
| CadFusion                         | 6.22           | 0.384               | 0.349                | 52.901             | 36.485              | 22.450              | 14.845               | 18.59             |
| <i>Qwen3.5-27B</i>                |                |                     |                      |                    |                     |                     |                      |                   |
| Vanilla                           | 5.93           | <u>0.376</u>        | 0.344                | 49.639             | 35.362              | 21.864              | <u>13.170</u>        | 52.00             |
| CoT                               | 7.11           | <b>0.390</b>        | 0.350                | 50.606             | 36.800              | 21.160              | 14.027               | 62.39             |
| CADCodeVerify                     | 6.69           | 0.375               | 0.349                | 51.004             | 37.668              | 21.879              | 14.224               | 13.55             |
| <b>Ours w/o Exp.</b>              | <u>7.56</u>    | <u>0.376</u>        | <b>0.363</b>         | <u>48.087</u>      | <u>33.296</u>       | <u>19.653</u>       | <b>12.425</b>        | 1.37              |
| <b>Ours</b>                       | <b>7.74</b>    | 0.370               | <u>0.355</u>         | <b>47.651</b>      | <b>32.116</b>       | <b>19.450</b>       | 13.228               | <u>1.16</u>       |
| <i>GPT-5</i>                      |                |                     |                      |                    |                     |                     |                      |                   |
| Vanilla                           | 7.46           | <b>0.411</b>        | <u>0.383</u>         | 49.564             | 33.473              | 19.677              | <u>11.478</u>        | 33.19             |
| CoT                               | 7.59           | <u>0.403</u>        | <b>0.391</b>         | 48.406             | <u>32.293</u>       | 19.383              | 12.282               | 27.42             |
| CADCodeVerify                     | 7.34           | 0.391               | 0.365                | 48.725             | 33.905              | 20.336              | 12.987               | 16.38             |
| <b>Ours w/o Exp.</b>              | <b>7.72</b>    | 0.398               | 0.375                | <b>46.325</b>      | <b>31.833</b>       | <b>18.874</b>       | <b>10.891</b>        | <u>0.11</u>       |
| <b>Ours</b>                       | <u>7.69</u>    | 0.395               | 0.367                | <u>47.636</u>      | 33.760              | <u>19.239</u>       | 11.735               | 0.32              |

Table 5: Main results on the full CADFUSION benchmark. Best and second-best results are shown in **bold** and underlined, respectively.

| Method                            | Text2CAD-Hard  |                     |                      |                    |                     |                     |                      |                   |
|-----------------------------------|----------------|---------------------|----------------------|--------------------|---------------------|---------------------|----------------------|-------------------|
|                                   | VLM $\uparrow$ | m. r-IoU $\uparrow$ | md. r-IoU $\uparrow$ | m. CD $\downarrow$ | md. CD $\downarrow$ | m. rCD $\downarrow$ | md. rCD $\downarrow$ | Inv. $\downarrow$ |
| <i>Trained Text-to-CAD Models</i> |                |                     |                      |                    |                     |                     |                      |                   |
| Text2CAD                          | 2.74           | 0.239               | 0.162                | 80.012             | 69.586              | 48.327              | 28.816               | <b>0.00</b>       |
| Cadrille                          | 1.38           | 0.038               | 0.013                | 170.415            | 151.596             | 92.485              | 87.336               | 3.00              |
| CAD-Coder                         | 2.94           | 0.306               | 0.281                | 66.779             | 49.126              | 44.868              | 27.259               | 3.00              |
| CadFusion                         | 5.05           | <b>0.354</b>        | <b>0.312</b>         | 55.036             | 42.341              | <b>26.226</b>       | <b>17.069</b>        | 14.00             |
| <i>Qwen3.5-27B</i>                |                |                     |                      |                    |                     |                     |                      |                   |
| Vanilla                           | 6.23           | 0.313               | 0.261                | 62.356             | 46.800              | 39.100              | 26.871               | 52.00             |
| CoT                               | 5.62           | 0.322               | 0.282                | 67.111             | 55.798              | 37.583              | 22.415               | 62.00             |
| CADCodeVerify                     | <u>6.29</u>    | 0.305               | 0.259                | 56.602             | 46.764              | 33.698              | 24.362               | 25.00             |
| <b>Ours w/o Exp.</b>              | 6.25           | 0.321               | 0.292                | <u>54.197</u>      | <u>38.604</u>       | 33.434              | 21.455               | <b>0.00</b>       |
| <b>Ours</b>                       | <b>6.54</b>    | <u>0.331</u>        | <u>0.306</u>         | <b>49.759</b>      | <b>35.803</b>       | <u>30.055</u>       | <u>19.411</u>        | <u>0.50</u>       |

Table 6: Supplementary results on TEXT2CAD-HARD for the robustness and experience-transfer analysis in Section 4.5. Best and second-best results are shown in **bold** and underlined.

| Method             | CadFusion-Hard |                     |                      |                    |                     |                     |                      |                   |
|--------------------|----------------|---------------------|----------------------|--------------------|---------------------|---------------------|----------------------|-------------------|
|                    | VLM $\uparrow$ | m. r-IoU $\uparrow$ | md. r-IoU $\uparrow$ | m. CD $\downarrow$ | md. CD $\downarrow$ | m. rCD $\downarrow$ | md. rCD $\downarrow$ | Inv. $\downarrow$ |
| <i>Qwen3.5-27B</i> |                |                     |                      |                    |                     |                     |                      |                   |
| Vanilla            | 4.68           | <b>0.356</b>        | 0.327                | 52.358             | 36.661              | 23.958              | <b>13.966</b>        | 50.50             |
| CAD Prompt         | 4.76           | 0.333               | 0.323                | <u>47.993</u>      | 37.835              | 24.506              | 17.266               | 11.00             |
| w/o SCC.+CSU       | 4.96           | 0.343               | <u>0.333</u>         | <b>46.638</b>      | <u>33.408</u>       | 22.320              | 15.342               | 3.50              |
| w/o SCC.           | 5.29           | 0.332               | <u>0.326</u>         | 54.636             | 35.848              | 23.883              | 16.889               | 3.00              |
| w/o CSU            | 5.18           | 0.345               | 0.332                | 51.741             | 34.648              | <b>21.015</b>       | 14.320               | 4.00              |
| <b>w/o Exp.</b>    | <u>5.67</u>    | 0.345               | <b>0.336</b>         | 49.682             | <b>32.589</b>       | 22.192              | 14.201               | <b>1.00</b>       |
| <b>Ours</b>        | <b>6.34</b>    | <u>0.347</u>        | 0.325                | 50.672             | 34.109              | <u>21.388</u>       | <u>14.193</u>        | <u>2.00</u>       |
| <i>GPT-5</i>       |                |                     |                      |                    |                     |                     |                      |                   |
| Vanilla            | 5.89           | 0.372               | 0.344                | 54.463             | 31.330              | 21.525              | 13.202               | 38.50             |
| CAD Prompt         | 5.4            | <u>0.380</u>        | <u>0.359</u>         | <b>45.882</b>      | 38.232              | 20.466              | 12.048               | 8.00              |
| w/o SCC+CSU        | 5.61           | <b>0.387</b>        | <b>0.374</b>         | 50.075             | 29.105              | 19.683              | 13.822               | 1.50              |
| w/o SCC.           | 5.92           | 0.379               | 0.349                | 49.270             | <b>28.043</b>       | <b>18.696</b>       | 12.936               | 2.50              |
| w/o CSU.           | 5.56           | 0.379               | 0.355                | 49.344             | 29.563              | 19.045              | <u>11.424</u>        | <b>0.00</b>       |
| <b>w/o Exp.</b>    | <u>6.16</u>    | 0.362               | 0.353                | 49.954             | 32.062              | 20.330              | 13.239               | <u>0.50</u>       |
| <b>Ours</b>        | <b>6.62</b>    | 0.373               | 0.352                | <u>47.157</u>      | <u>28.680</u>       | <u>19.307</u>       | <b>11.032</b>        | <u>0.50</u>       |

Table 7: Ablation study on CADFUSION-HARD. **Vanilla** uses a generic prompt. **CAD Prompt** denotes our CAD-tailored prompt design without the full structured pipeline. **Ours** is the full framework. **w/o Exp.** removes reusable experience for constraint completion. **w/o SCC** removes spatial constraint completion. **w/o CSU** removes construction structure understanding. **w/o SCC+CSU** removes both spatial constraint completion and construction structure understanding. Best and second-best results are shown in **bold** and underlined, respectively.

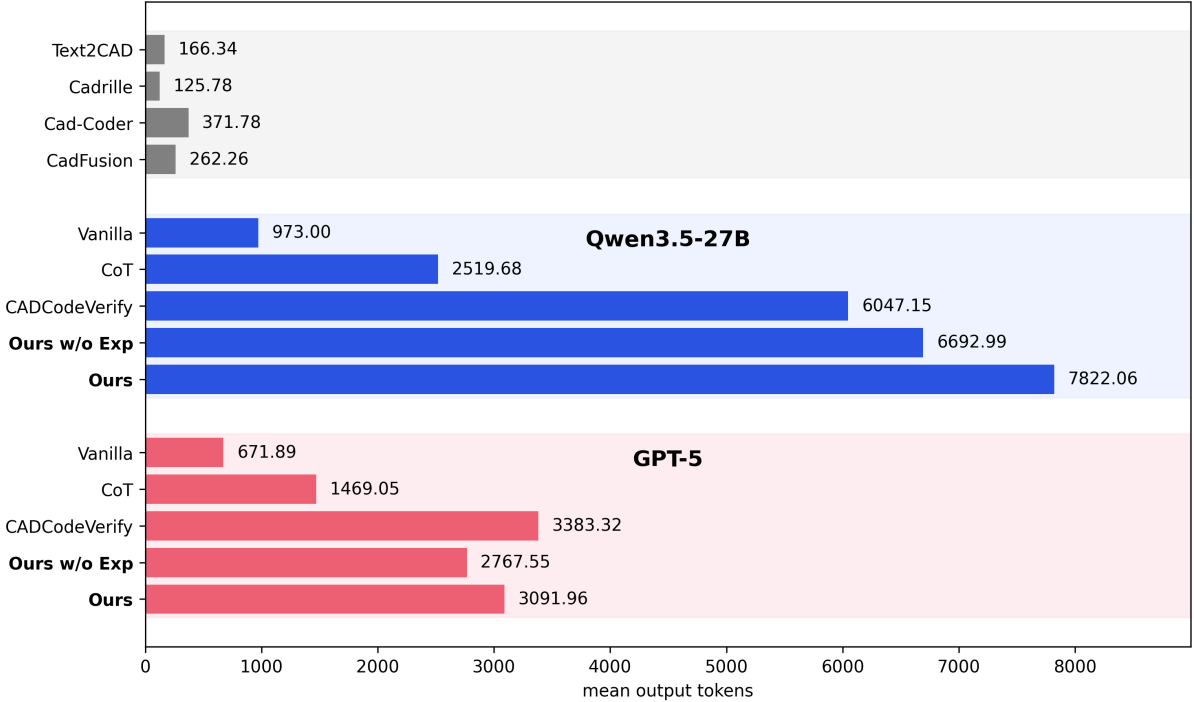


Figure 17: Comparison of mean output tokens.